Plan de verificación: UART 16550

Llenen todas las tablas. Las filas en gris son ejemplos que muestran el nivel de detalle esperado: reemplácenlas por las suyas. Pueden agregar filas y secciones, pero no eliminar ninguna.

Portada

Campo	Valor
Proyecto	Verificación del UART 16550 con UVM
Integrante 1	Luis Diego García Rojas
Integrante 2	Eric Jossué Castro Velásquez
Integrante 3	Randy Steve Fernández Aguilar
Versión del plan	v1
Fecha	29 de octubre del 2026
Repositorio Git	https://github.com/Scrys26/Proyecto2-SystemVerilog-Verification-
Etiqueta Git	N/A

1. Alcance

Campo	Valor
Versión del RTL verificada (commit o revisión SVN)	N/A
Configuración de compilación (bus de 32 bits; variante de 8 bits sí/no)	bus de 32 bits
Frecuencia de reloj en simulación y rango de DL soportado	N/A
Funcionalidad fuera de alcance y justificación	N/A

2. Documentos de referencia

UART_spec	Versión	Enlace o ubicación
UART IP Core Specification	*Rev. 0.6*	*doc/UART_spec.pdf*

3. Interpretación de la especificación

Una fila por cada ambigüedad o discrepancia entre la especificación y el RTL (sección 3.5 del enunciado).

ID	Tema	La especificación dice	El RTL hace	Comportamiento que modelamos	Justificación
AMB-01	Registro scratch en la dirección 7	En la sección operación se menciona que el scratch fue removido	En el RTL en el módulo uart_regs.v se agrega en la dirección 7 el registro scratch, que puede leerse y escribirse y se reinicia a cero.	Lo modelaremos como un registro adicional de 8 bits, de lectura y escritura. Se identificará como una funcionalidad implementada por el RTL, pero no exigida por la especificación.	Aunque la especificación indica que fue eliminado, el RTL lo implementa. Se documentará como una diferencia y se comprobará su funcionamiento sin considerarlo una funcionalidad obligatoria de la especificación.
AMB-02	Lectura del registro MCR	La sección 4.1 y 4.6 definen el MCR, dirección 4 como un registro de solo escritura	uart_regs.v permite escribir en MCR, pero no contempla su de dirección en el multiplexor de lectura.	MCR se tomará únicamente como registro de escritura. Además, no se recuperará su valor mediante lectura normal.	El RTL coincide con el acceso de solo escritura documentado. El valor cero observado al leer no debe confundirse con el contenido almacenado en MCR.

ID	Tema	La especificación dice	El RTL hace	Comportamiento que modelamos	Justificación
AMB-03	Polaridad de las señales de módem	La sección 4.6 establece que los bits DTR y RTS del MCR controlan salidas invertidas: un bit en 0 produce salida 1 y viceversa. La sección 4.8 define los bits superiores del MSR como complementos de las entradas de módem.	uart_regs.v asigna directamente los bits 0 y 1 del MCR a rts_pad_o y dtr_pad_o. Además, fuera de loopback, los bits superiores de MSR [7:4] reciben directamente las señales físicas de entrada.	En primera instancia tomaremos como referencias las polaridades de la especificación, diferenciado las señale lógicas internas de los niveles físicos de los pines.	Existe una diferencia entre la polaridad indicada para DTR/RTS y la asignación física del RTL. En MSR, la inversión interna debe considerarse antes de declarar una discrepancia.
AMB-04	Detección de TERI	La sección 4.8 indica que TERI (MSR [2]) se activa cuando RI cambia de nivel bajo a alto.	uart_regs. v utiliza una operación XOR entre las señales actuales y las anteriores, detectando cualquier cambio de RI, tanto de 0 a 1 como de 1 a 0.	Se implementará la activación de TERI ante la transición de RI de 0 a 1, según la especificación. Se aplicarán ambas transiciones para comprobar el comportamiento implementado.	La especificación establece una transición específica, mientras que el RTL detecta cualquier cambio de nivel. Esta diferencia debe documentarse y comprobarse durante la verificación.
AMB-05	Selección de bytes mediante wb_sel_i	En la sección 5 se establece que wb_sel_i permite seleccionar bytes en las transferencias Wishbone de 32 bits. No se define qué debe pasa ante de combinaciones de selección inválidas.	Uart_wb.v utiliza las selecciones one-hot para escoger el byte y la dirección interna. En casos no reconocidos, selecciona por defecto los bits [7:0] para escritura y la dirección inferior 0. Para lectura, esas selecciones devuelven cero, excepto 1111, la cual se utiliza para la depuración.	Se usarán selecciones one-hot para los accesos normales de 8 bits y 1111 para lecturas de depuración de 32 bits. Las demás combinaciones se clasificarán como comportamientos no especificados.	No se deben asumir resultados funcionales para selecciones que la especificación no define. Se documentará el comportamiento del RTL y se evitará interpretar los accesos inesperados como operaciones válidas del protocolo.
AMB-06	Generación y recepción de bits de parada.	La sección 4.5 establece 1 bits de parada, 1.5 bits para caracteres de 5 bits y 2 bits para caracteres de 6 a 8 bits. El receptor únicamente comprueba el primer bit de parada.	uart_transmitter.sv contiene lógica para seleccionar las tres duraciones uart_receiver.v comprueba un único bit de parada. Sin embargo, CHANGES.txt contiene una nota histórica que indica que más de un bit de parada no estaba implementado.	Se utilizarán 3 configuraciones de transmisión y se comprobará su duración. En la recepción se comprobará únicamente el primer bit de parada, conforme a la especificación.	El RTL actual contiene lógica para las tres configuraciones. La limitación mencionada en CHANGES.txt corresponde a una versión anterior y no debe asumirse vigente. La duración exacta deberá confirmarse mediante simulación.

4. Lista de características

ID	Característica	Fuente (sección de la especificación)	Prioridad (alta, media, baja)
F-GEN-01	*Bus de datos de 32 bits (modo por defecto y obligatorio)*	*Introduction*	*Alta*
F-GEN-02	*Operación solo en modo FIFO*	*Introduction*	*Alta*
F-GEN-03	*Variante de bus de 8 bits (DATA_BUS_WIDTH_8; puntos extra)*	*Introduction*	*Baja*
F-RST-01	*Reset asíncrono*	*WISHBONE interface signals, WB_RST_I*	*Alta*
F-WB-01	*Selección de registro con WB_ADDR_I (5 bits en 32 bits, 3 bits en 8 bits)*	*WISHBONE interface signals*	*Alta*
F-WB-02	*Selección de byte con WB_SEL_I*	*WISHBONE interface signals*	*Alta*
F-WB-03	*Ciclos de lectura y escritura Wishbone terminados con ACK*	*WISHBONE interface signals*	*Alta*
F-IO-01	*Salida de interrupción INT_O*	*Other internal signals*	*Alta*
F-IO-02	*Salida opcional BAUD_O a 16 × baudios*	*Other internal signals*	*Baja*
F-IO-03	*Línea serie: salida STX_PAD_O y entrada SRX_PAD_I*	*External (off-chip) connections*	*Alta*
F-IO-04	*Salidas de módem RTS y DTR; entradas CTS, DSR, RI y DCD*	*External (off-chip) connections*	*Media*
F-IO-05	*INT_O refleja las interrupciones habilitadas y pendientes*	*Other internal signals, Interrupt Enable Register, Interrupt Identification Register*	*Alta*
F-REG-01	*Mapa de registros: dirección, ancho y sentido de acceso*	*Registers list*	*Alta*
F-REG-02	*Registros distintos para leer y escribir en la misma dirección (RBR/THR, IIR/FCR)*	*Registers list*	*Alta*
F-REG-03	*Divisor accesible en las direcciones 0 y 1 solo con DLAB = 1; con DLAB = 1 no se puede acceder a RBR, THR ni IER*	*Registers list*	*Alta*
F-REG-04	*Registros Debug 1 y 2 disponibles solo en modo de 32 bits*	*Registers list*	*Media*
F-IER-01	*Habilitación de la interrupción de dato recibido*	*Interrupt Enable Register, bit 0*	*Alta*
F-IER-02	*Habilitación de la interrupción THRE*	*Interrupt Enable Register, bit 1*	*Alta*
F-IER-03	*Habilitación de la interrupción de estado de línea*	*Interrupt Enable Register, bit 2*	*Alta*
F-IER-04	*Habilitación de la interrupción de módem*	*Interrupt Enable Register, bit 3*	*Media*
F-IER-05	*Bits 7-4 reservados en 0; reset 00h*	*Interrupt Enable Register, bits 7-4*	*Baja*
F-INT-01	*Bit 0 = 0 indica que hay una interrupción pendiente*	*Interrupt Identification Register, bit 0*	*Alta*
F-INT-02	*IIR muestra la interrupción pendiente de mayor prioridad*	*Interrupt Identification Register, bits 3-1*	*Alta*
F-INT-03	*Interrupción RLS ante errores de paridad, overrun, framing o break*	*Interrupt Identification Register*	*Alta*
F-INT-04	*Interrupción RDA al alcanzar el nivel de disparo de la FIFO RX*	*Interrupt Identification Register*	*Alta*
F-INT-05	*Interrupción de timeout tras 4 tiempos de carácter*	*Interrupt Identification Register*	*Alta*
F-INT-06	*Interrupción THRE cuando THR está vacío*	*Interrupt Identification Register*	*Alta*
F-INT-07	*Interrupción del Modem Status ante cambios en CTS, DSR, RI o DCD*	*Interrupt Identification Register*	*Alta*

ID	Característica	Fuente (sección de la especificación)	Prioridad (alta, media, baja)
F-INT-08	*Bits 5-4 en 0 y bits 7-6 en 1; reset C1h*	*Interrupt Identification Register, bits 7-4*	*Baja*
F-INT-09	*RLS se limpia al leer LSR*	*Interrupt Identification Register*	*Alta*
F-INT-10	*RDA se limpia cuando la FIFO baja del nivel de disparo*	*Interrupt Identification Register*	*Alta*
F-INT-11	*Timeout se limpia al leer RBR*	*Interrupt Identification Register*	*Alta*
F-INT-12	*THRE se limpia al escribir THR o al leer IIR*	*Interrupt Identification Register*	*Alta*
F-INT-13	*Modem Status se limpia al leer MSR*	*Interrupt Identification Register*	*Alta*
F-INT-14	*Orden de prioridad: RLS (4'h6), luego RDA (4'h4) o timeout (4'hC), luego THRE (4'h2), luego módem (4'h0)*	*Interrupt Identification Register*	*Alta*
F-FCR-01	*Bit 0 ignorado (siempre en modo FIFO)*	*FIFO Control Register, bit 0*	*Baja*
F-FCR-02	*Limpieza de la FIFO RX sin afectar el registro de desplazamiento*	*FIFO Control Register, bit 1*	*Media*
F-FCR-03	*Limpieza de la FIFO TX sin afectar el registro de desplazamiento*	*FIFO Control Register, bit 2*	*Media*
F-FCR-04	*Bits 5-3 ignorados*	*FIFO Control Register, bits 5-3*	*Baja*
F-FCR-05	*Nivel de disparo de la FIFO RX en 1, 4, 8 o 14 bytes*	*FIFO Control Register, bits 7-6*	*Alta*
F-FCR-06	*Reset 11000000b (FCR es de solo escritura: no se lee por puerta delantera)*	*FIFO Control Register*	*Media*
F-LCR-01	*Longitud de carácter de 5, 6, 7 y 8 bits*	*Line Control Register, bits 1-0*	*Alta*
F-LCR-02	*1, 1.5 o 2 bits de parada (1.5 solo con 5 bits; 2 con las demás longitudes); RX revisa solo el primero*	*Line Control Register, bit 2*	*Alta*
F-LCR-03	*Paridad par o impar; TX lo genera y RX revisa*	*Line Control Register, bits 3 y 4*	*Alta*
F-LCR-04	*Paridad stick (forzada a 0 o 1)*	*Line Control Register, bit 5*	*Media*
F-LCR-05	*Break en la transmisión (stx forzado a 0)*	*Line Control Register, bit 6*	*Media*
F-LCR-06	*DLAB (LCR[7]): en 1 se accede a los divisores, en 0 a los registros normales; se restaura a 0 tras escribir el divisor*	*Line Control Register, bit 7*	*Media*
F-LCR-07	*LCR se puede leer; reset 00000011b*	*Line Control Register*	*Media*
F-MCR-01	*Control de DTR (bit en 1 → DTR en 0)*	*Modem Control Register, bit 0*	*Alta*
F-MCR-02	*Control de RTS (bit en 1 → RTS en 0)*	*Modem Control Register, bit 1*	*Alta*
F-MCR-03	*Out1 conectado a RI en loopback*	*Modem Control Register, bit 2*	*Media*
F-MCR-04	*Out2 conectado a DCD en loopback*	*Modem Control Register, bit 3*	*Media*
F-MCR-05	*Loopback: STX_PAD_O en 1 y transmisor conectado internamente al receptor*	*Modem Control Register, bit 4*	*Alta*
F-MCR-06	*Loopback de módem: DTR→DSR, RTS→CTS, Out1→RI, Out2→DCD*	*Modem Control Register, bit 4*	*Media*
F-MCR-07	*Solo escritura (no se puede leer de vuelta); bits 7-5 ignorados; reset 0*	*Modem Control Register, bits 7-5*	*Media*
F-LSR-01	*Data Ready: hay al menos un carácter en la FIFO RX*	*Line Status Register, bit 0*	*Alta*
F-LSR-02	*Overrun: FIFO llena y llega otro carácter; la FIFO se conserva; se limpia al leer*	*Line Status Register, bit 1*	*Alta*
F-LSR-03	*Error de paridad del carácter al frente de la FIFO; se limpia al leer*	*Line Status Register, bit 2*	*Alta*
F-LSR-04	*Error de trama (stop bit inválido) del*	*Line Status Register, bit 3*	*Alta*

ID	Característica	Fuente (sección de la especificación)	Prioridad (alta, media, baja)
	carácter al frente de la FIFO; se limpia al leer		
F-LSR-05	*Break: línea en 0 por un tiempo de carácter; entra un carácter 0 a la FIFO; se limpia al leer*	*Line Status Register, bit 4*	*Alta*
F-LSR-06	*FIFO TX vacía; genera THRE; se limpia al escribir THR*	*Line Status Register, bit 5*	*Alta*
F-LSR-07	*FIFO TX y registro de desplazamiento vacíos; se limpia al escribir THR*	*Line Status Register, bit 6*	*Alta*
F-LSR-08	*Hay al menos un error de paridad, de trama o un break dentro de la FIFO; se limpia al leer*	*Line Status Register, bit 7*	*Media*
F-MSR-01	*DCTS: CTS cambió de estado*	*Modem Status Register, bit 0*	*Media*
F-MSR-02	*DDSR: DSR cambió de estado*	*Modem Status Register, bit 1*	*Media*
F-MSR-03	*TERI: RI pasó de bajo a alto*	*Modem Status Register, bit 2*	*Alta*
F-MSR-04	*DDCD: DCD cambió de estado*	*Modem Status Register, bit 3*	*Media*
F-MSR-05	*Los bits de cambio se limpian al leer MSR*	*Modem Status Register, bits 3-0*	*Media*
F-MSR-06	*Complemento de CTS, DSR, RI y DCD, o RTS, DTR, Out1 y Out2 en loopback*	*Modem Status Register, bits 7-4*	*Media*
F-DL-01	*Divisor de 16 bits formado por dos bytes*	*Divisor Latches*	*Alta*
F-DL-02	*Reset en 0, que deshabilita toda la E/S serie*	*Divisor Latches*	*Media*
F-DL-03	*Tasa de baudios = reloj / (16 × DL)*	*Divisor Latches*	*Alta*
F-DL-04	*El contador interno arranca al escribir el byte bajo (DLL)*	*Divisor Latches*	*Media*
F-DBG-01	*Debug 1 de solo lectura (LSR, IER, IIR, LCR y MSR); leerlo no altera el núcleo*	*Debug 1*	*Media*
F-DBG-02	*Debug 2 de solo lectura (estados de las FSM, conteo de FIFOs, MCR y FCR); leerlo no altera el núcleo*	*Debug 2*	*Media*
F-OP-01	*El RTL implementa un registro Scratch de 8 bits en la dirección interna 7, con acceso de lectura y escritura y valor de reset 0x00, aunque la especificación indica que fue eliminado.*	*Operation*	*Media*
F-INI-01	*Con el reset se vacían las FIFOs RX y TX*	*Initialization*	*Alta*
F-INI-02	*Con el reset se limpian los registros de desplazamiento de RX y TX*	*Initialization*	*Media*
F-INI-03	*Secuencia de inicialización: LCR con DLAB, DLM, DLL, DLAB = 0, FCR, IER*	*Initialization*	*Alta*
F-INI-04	*Muestreo a mitad de bit, tolerante a pequeñas diferencias de reloj*	*Initialization*	*Media*
F-TX-01	*Una escritura a THR entra a la FIFO TX y se transmite por STX_PAD_O con el formato de LCR y la tasa de DL*	*Registers list (THR), Line Control Register, Divisor Latches*	*Alta*
F-TX-02	*FIFO TX de 16 posiciones: llenado y escritura con la FIFO llena*	*Enunciado 4.1*	*Alta*
F-RX-01	*Lo recibido por SRX_PAD_I entra a la FIFO RX y sale en orden por RBR; leer RBR extrae el dato*	*Registers list (Receiver Buffer)*	*Alta*

5. Matriz de verificación

Una o más filas por cada característica de la sección 4.

ID ítem	Característica	Qué se verifica	Mecanismo (scoreboard, SVA, RAL, cobertura)	Métrica de cierre (coverpoint o aserción)	Plusargs que lo dirigen
V-GEN-01	*F-GEN-01*	*Compilación en modo de 32 bits (sin DATA_BUS_WIDTH_8): dirección de 5 bits, wb_sel_i activo y Debug 1/2 accesibles*	*RAL (puerta delantera); chequeo en compilación (`ifdef` con $error)*	*cg_regs.cp_addr (incluye 8 y 12)*	*Ninguno (define de compilación; es el modo por defecto)*
V-GEN-02	*F-GEN-02*	*El UART opera siempre con FIFO: FCR[0] no cambia el comportamiento*	*Scoreboard (modelo con FIFO siempre activa)*	*cg_fifo_cfg.cp_fcr_b0 × cg_tx.cp_tx_fifo_level[2:16]*	*+UART_FCR_IGN_BITS_PCT*
V-GEN-03	*F-GEN-03*	*Variante de 8 bits (extra): mismos chequeos con bus de 8 bits y dirección de 3 bits*	*Compilación con +define+DATA_BUS_WIDTH_8; driver y adaptador parametrizados por wb_agent_cfg*	*cg_regs.cp_addr[0:6] en la regresión de la variante*	*+UART_WB_BUS_WIDTH=8 (junto con el define)*
V-RST-01	*F-RST-01*	*Un reset asíncrono en cualquier momento deja el DUT en el estado de §5.1 y el modelo se resincroniza*	*Scoreboard (vaciado del modelo); RAL (uvm_reg_hw_reset_seq); SVA*	*cg_rst.cp_reset_phase (reposo, TX activo, RX activo, FIFO llena, DLAB = 1); A-RST-01*	*+UART_RST_EN; +UART_RST_MID_TRAFFIC_PCT*
V-WB-01	*F-WB-01*	*Cada dirección decodifica al registro correcto en 32 bits (5 bits de dirección)*	*RAL (uvm_reg_access_seq y bit_bash por puerta delantera); scoreboard de lecturas*	*cg_regs.cp_addr × cp_dir × cp_dlab*	*+UART_REG_ADDR_WEIGHTS; +UART_REG_DLAB_PCT*
V-WB-02	*F-WB-02*	*wb_sel_i con un solo bit activo selecciona el byte correcto en lectura y escritura*	*Adaptador reg2bus/bus2reg; scoreboard; SVA*	*cg_bus.cp_sel (4'b0001, 0010, 0100, 1000) × cp_dir; A-WB-01*	*+UART_WB_SEL_WEIGHTS*
V-WB-02b	*F-WB-02*	*Comportamiento con wb_sel_i de cero bits o más de un bit activo (punto 3.5; la especificación no lo define)*	*Monitor Wishbone; se registra el resultado como ambigüedad (no se compara contra la especificación)*	*cg_bus.cp_sel_illegal (0 bits, 2 o más bits)*	*+UART_WB_SEL_ILLEGAL_PCT (por defecto 0)*
V-WB-03	*F-WB-03*	*Cada ciclo cyc && stb termina con ack en un tiempo acotado, sin X y con señales estables*	*SVA; monitor Wishbone*	*A-WB-01, A-WB-02, A-WB-03, A-WB-04 (assert y cover); cg_bus.cp_ack_latency*	*+UART_WB_DELAY_MAX; +UART_WB_STB_DELAY_MAX*
V-WB-04	*F-WB-01*	*Accesos a direcciones no definidas (9 a 11 y 13 a 31 en modo de 32 bits): la especificación no define la respuesta; se registra si hay ack, qué dato se lee y si hay efectos. La dirección 7 va en V-OP-01.*	*Monitor Wishbone; scoreboard sin comparación (se documenta como ambigüedad); SVA de ack acotado*	*cg_regs.cp_addr[no mapeadas] × cp_dir (ya en 6.1 y 6.2)*	*+UART_REG_ADDR_WEIGHTS*
V-LSR-04	*F-LSR-04*	*FE = 1 para el carácter al frente de la FIFO sin bit de parada válido; se limpia al leer LSR.*	*Scoreboard (LSR); agente serie (inyección); RAL (callback)*	*cg_err.cp_fe × cg_line_cfg.cp_wls*	*+UART_SER_FRAMING_ERR_PCT*
V-LSR-05	*F-LSR-05*	*BI = 1 cuando la línea queda en 0 por un*	*Scoreboard (LSR y FIFO RX); agente*	*cg_err.cp_bi × cp_break_len*	*+UART_SER_BREAK_PCT,*

ID ítem	Característica	Qué se verifica	Mecanismo (scoreboard, SVA, RAL, cobertura)	Métrica de cierre (coverpoint o aserción)	Plusargs que lo dirigen
		tiempo de carácter completo; entra un carácter 0 a la FIFO; se limpia al leer LSR.	*serie (inyección); RAL (callback)*		*+UART_SER_BREAK_LEN_MAX*
V-LSR-06	*F-LSR-06*	*LSR[5] = 1 con la FIFO TX vacía y vuelve a 0 al escribir THR; genera la interrupción THRE.*	*Scoreboard (LSR y FIFO TX); RAL*	*cg_tx.cp_lsr_thre* × cp_tx_fifo_level*	*+UART_TX_BURST_MIN, +UART_TX_BURST_MAX*
V-LSR-07	*F-LSR-07*	*LSR[6] = 1 solo si la FIFO TX y el registro de desplazamiento están vacíos; con el último carácter en la línea, THRE puede valer 1 y TEMT 0.*	*Scoreboard (LSR y FIFO TX); Debug 2 (tf_count)*	*cg_tx.cp_lsr_temt* × cp_tx_busy*	*+UART_TX_BURST_MAX*
V-LSR-08	*F-LSR-08*	*LSR[7] = 1 mientras haya al menos un carácter con error de paridad, trama o break dentro de la FIFO; se limpia al leer LSR.*	*Scoreboard (LSR); RAL (callback)*	*cg_err.cp_lsr7* × cp_err_in_fifo**	*+UART_SER_PARITY_ERR_PCT, +UART_SER_FRAMING_ERR_PCT, +UART_SER_BREAK_PCT*
V-DBG-01	*F-DBG-01*	*Debug 1 (dirección 8, wb_sel_i = 1111) = {MSR [31:24], LCR [23:16], IIR[3:0] [15:12], IER[3:0] [11:8], LSR [7:0]}; leerlo no limpia LSR ni cambia el estado.*	*Scoreboard (comparación con el modelo); RAL (registro de solo lectura sin efectos laterales)*	*cg_regs.cp_addr[8] × cp_dbg_lsr_err_pending**	*+UART_REG_ADDR_WEIGHTS*
V-DBG-02	*F-DBG-02*	*Debug 2 (dirección 12): tf_count [7:3] y rf_count [16:12] iguales al modelo de FIFO; MCR y FCR reflejados; los estados de FSM [2:0] y [11:8] solo se observan, sin modelo. Los anchos de MCR [18:17] y FCR [23:19] no cuadran en la especificación (ambigüedad para la sección 3).*	*Scoreboard (conteos); RAL; Debug 2 ya se usa en V-FCR-02, V-FCR-03 y V-INI-01*	*cg_regs.cp_addr[12] × cp_fifo_count**	*+UART_REG_ADDR_WEIGHTS*
V-OP-01	*F-OP-01*	*Se comprueba que el registro Scratch de la dirección interna 7 permite escribir y leer valores de 8 bits, y que después del reset retorna 0x00. El comportamiento se documenta como una extensión del RTL no requerida por la especificación.*	*RAL (registro Scratch de la implementación); scoreboard de registros; monitor Wishbone; comprobación de reset.*	*cg_regs.cp_addr[7] × cp_dir × cp_dlab; comprobación de reset de Scratch.*	*+UART_REG_ADDR_WEIGHTS*
V-REG-01	*F-REG-01*	*Lectura y escritura en cada dirección (0 a 7, 8 y 12) con el sentido de acceso correcto; el valor leído coincide con el espejo del RAL*	*RAL (puerta delantera y trasera, uvm_reg_access_seq)*	*cg_regs.cp_addr × cp_dir*	*+UART_REG_ADDR_WEIGHTS*
V-REG-02	*F-REG-02*	*Escribir en la dirección 0 mete un byte en la FIFO TX y leerla saca un byte de la FIFO RX; escribir en la dirección 2 cambia*	*Scoreboard; RAL*	*cg_regs.cp_addr[0, 2] × cp_dir*	*+UART_REG_ADDR_WEIGHTS*

ID ítem	Característica	Qué se verifica	Mecanismo (scoreboard, SVA, RAL, cobertura)	Métrica de cierre (coverpoint o aserción)	Plusargs que lo dirigen
		FCR y leerla devuelve IIR			
V-REG-03	*F-REG-03*	*Con DLAB = 1, las direcciones 0 y 1 acceden a DLL y DLM (regs.dl por puerta trasera) sin tocar las FIFOs ni IER; con DLAB = 0, al revés*	*RAL (mapas DLAB 0 y 1, puerta trasera)*	*cg_regs.cp_addr × cp_dir × cp_dlab*	*+UART_REG_DLAB_PCT*
V-REG-04	*F-REG-04*	*En modo de 32 bits, las direcciones 8 y 12 con wb_sel_i = 1111 devuelven Debug 1 y Debug 2*	*RAL (puerta delantera); scoreboard*	*cg_regs.cp_addr[8, 12]*	*+UART_REG_ADDR_WEIGHTS*
V-FCR-01	*F-FCR-01*	*Escribir FCR[0] en 0 o en 1 no cambia el comportamiento de las FIFOs*	*Scoreboard (modelo con FIFO siempre activa)*	*cg_fifo_cfg.cp_fcr_b0*	*+UART_FCR_IGN_BITS_PCT*
V-FCR-02	*F-FCR-02*	*FCR[1] = 1 vacía la FIFO RX (LSR[0] = 0, rf_count = 0 en Debug 2) y el carácter en curso se recibe completo*	*Scoreboard (modelo de FIFO); Debug 2*	*cg_fifo_cfg.cp_rx_clr × cp_rx_busy*	*+UART_FCR_CLR_PCT*
V-FCR-03	*F-FCR-03*	*FCR[2] = 1 vacía la FIFO TX y el carácter en curso termina de transmitirse completo*	*Scoreboard (modelo de FIFO); Debug 2*	*cg_fifo_cfg.cp_tx_clr × cp_tx_busy*	*+UART_FCR_CLR_PCT*
V-FCR-04	*F-FCR-04*	*Valores aleatorios en FCR[5:3] no cambian el comportamiento*	*Scoreboard*	*cg_fifo_cfg.cp_fcr_b5_3*	*+UART_FCR_IGN_BITS_PCT*
V-FCR-05	*F-FCR-05*	*Los cuatro niveles de disparo (1, 4, 8 y 14) se configuran y la FIFO RX llega a cada uno; la generación de RDA se verifica en V-INT-04*	*Scoreboard (modelo de FIFO)*	*cg_fifo_cfg.cp_tl × cg_rx.cp_rx_fifo_level*	*+UART_FCR_TL_WEIGHTS, +UART_SW_SERVICE_DELAY_MAX*
V-FCR-06	*F-FCR-06*	*Tras el reset, regs.fcr = 2'b11 y RDA se activa a los 14 bytes sin haber escrito FCR*	*RAL (puerta trasera); scoreboard*	*uvm_reg_hw_reset_seq sin errores*	*Ninguno específico; configuración base*
V-LCR-01	*F-LCR-01*	*Datos TX y RX correctos con cada longitud*	*Scoreboard TX y RX*	*cg_line_cfg.cp_wls × cp_dir*	*+UART_LCR_WLS_WEIGHTS*
V-LCR-02	*F-LCR-02*	*TX genera 1, 1.5 o 2 bits de parada con la duración correcta*	*Monitor TX; scoreboard*	*cg_line_cfg.cp_stb × cp_wls*	*+UART_LCR_STB_PCT*
V-LCR-03	*F-LCR-02*	*Un 0 en el segundo bit de parada no produce error de trama (RX revisa solo el primero)*	*Scoreboard RX (LSR[3] = 0)*	*cg_err.cp_stop2_err*	*+UART_SER_STOP2_ERR_PCT*
V-LCR-04	*F-LCR-03*	*El bit de paridad en TX es correcto; RX*	*Scoreboard TX y RX*	*cg_line_cfg.cp_parity[ODD, EVEN] ×*	*+UART_LCR_PARITY_WEIGHTS*

ID ítem	Característica	Qué se verifica	Mecanismo (scoreboard, SVA, RAL, cobertura)	Métrica de cierre (coverpoint o aserción)	Plusargs que lo dirigen
		marca PE solo si se corrompió		*cp_dir*	
V-LCR-05	*F-LCR-04*	*Paridad stick 0 y stick 1 correcta en TX y RX*	*Scoreboard TX y RX*	*cg_line_cfg.cp_parity[STICK0, STICK1] × cp_dir*	*+UART_LCR_PARITY_WEIGHTS*
V-LCR-06	*F-LCR-05*	*Con LCR[6] = 1, stx = 0 mientras dure; al quitarlo, vuelve a 1*	*SVA (A-SER-04); monitor TX*	*cg_line_cfg.cp_bc; cover de A-SER-04*	*+UART_LCR_BC_PCT, +UART_LCR_BC_LEN_MAX*
V-LCR-07	*F-LCR-06*	*Con DLAB = 1 se accede a los divisores; con DLAB = 0 se accede a RBR/THR e IER*	*RAL (puerta delantera contra regs.dl por puerta trasera)*	*cg_regs.cp_addr × cp_dlab*	*+UART_REG_DLAB_PCT*
V-LCR-08	*F-LCR-07*	*LCR vale 0x03 tras el reset y se lee igual a lo que se escribió*	*RAL (uvm_reg_hw_reset_seq, uvm_reg_access_seq)*	*Secuencias del RAL sin errores*	*Ninguno específico; configuración base*
V-DL-01	*F-DL-01*	*Escribir DLM y DLL por separado forma regs.dl = {DLM, DLL} y se leen de vuelta igual*	*RAL (puerta delantera y trasera)*	*cg_line_cfg.cp_dl*	*+UART_LCR_DL_MAX*
V-DL-02	*F-DL-02*	*Con DL = 0 (tras el reset o escrito) no sale nada por stx y no entra nada a la FIFO RX*	*SVA (A-SER-05); scoreboard*	*cg_line_cfg.cp_dl[ZERO]*	*+UART_LCR_DL_ZERO_PCT*
V-DL-03	*F-DL-03*	*Cada bit en stx dura 16 × DL ciclos de reloj y RX recibe bien con el mismo DL*	*Monitor TX; SVA (A-SER-02)*	*cg_line_cfg.cp_dl × cp_wls*	*+UART_LCR_DL_MIN, +UART_LCR_DL_MAX_FAST, +UART_LCR_DL_MAX, +UART_LCR_DL_TAIL_PCT*
V-DL-04	*F-DL-04*	*Escribir solo DLM no cambia la tasa; la nueva tasa aplica después de escribir DLL*	*Monitor TX; scoreboard*	*cg_reconfig.cp_dl_order*	*+UART_CFG_RECONFIG_PCT*
V-INI-01	*F-INI-01*	*Tras el reset (inicial o aleatorio), las FIFOs quedan vacías: LSR = 0x60 y rf_count = tf_count = 0 en Debug 2*	*RAL (uvm_reg_hw_reset_seq, puerta trasera); SVA (A-RST-01)*	*cg_rst.cp_reset_phase*	*+UART_RST_EN, +UART_RST_MID_TRAFFIC_PCT*
V-INI-02	*F-INI-02*	*Un reset en medio de TX o RX deja stx en 1 de inmediato y el carácter parcial no entra a la FIFO*	*SVA; scoreboard*	*cg_rst.cp_reset_phase[TX activo, RX activo]*	*+UART_RST_EN, +UART_RST_MID_TRAFFIC_PCT*
V-INI-03	*F-INI-03*	*La secuencia de inicialización recomendada deja al UART transmitiendo y recibiendo con la configuración pedida*	*Scoreboard; RAL*	*cg_regs.cp_init_seq*	*+UART_CFG_RECONFIG_PCT*

ID ítem	Característica	Qué se verifica	Mecanismo (scoreboard, SVA, RAL, cobertura)	Métrica de cierre (coverpoint o aserción)	Plusargs que lo dirigen
V-INI-04	*F-INI-04*	*Con desviación de baudios dentro de ±3 % se recibe bien; por encima de ±6 % hay error de trama o dato corrupto*	*Scoreboard RX*	*cg_ser.cp_baud_dev*	*+UART_SER_BAUD_DEV_PCT, +UART_SER_BAUD_DEV_MAX*
V-LSR-01	*F-LSR-01*	*LSR[0] (Data Ready) vale 1 mientras la FIFO RX tiene al menos un carácter y 0 cuando está vacía*	*Scoreboard (modelo de FIFO RX); RAL (lectura de LSR)*	*cg_rx.cp_rx_fifo_level[0:16]*	*+UART_SW_SERVICE_DELAY_MAX, +UART_FCR_CLR_PCT*
V-LSR-02	*F-LSR-02*	*LSR[1] (Overrun) vale 1 cuando llega un carácter con la FIFO RX llena; la FIFO conserva su contenido y el bit se limpia al leer LSR*	*Scoreboard (modelo de FIFO RX con bandera de overrun); RAL*	*cg_err.cp_oe × cp_tl*	*+UART_SW_SERVICE_DELAY_MIN, +UART_FCR_TL_WEIGHTS*
V-LSR-03	*F-LSR-03*	*LSR[2] (Parity Error) vale 1 cuando el carácter al frente de la FIFO RX tiene error de paridad y se limpia al leer LSR; no se activa si la paridad es correcta*	*Scoreboard RX (modelo de FIFO con banderas de error por carácter); RAL*	*cg_err.cp_pe × cg_line_cfg.cp_parity*	*+UART_SER_PARITY_ERR_PCT, +UART_LCR_PARITY_WEIGHTS*
V-INI-05	*F-INI-04*	*Un glitch más corto que medio bit en srx no genera un carácter*	*Scoreboard RX*	*cg_ser.cp_glitch*	*+UART_SER_GLITCH_PCT*
V-IER-01	*F-IER-01*	*IER [0] habilita o enmascara las interrupciones RDA y timeout. Se comprueba que ambas fuentes sean ejercitadas con el bit 0 y en 1.*	*RAL; scoreboard de interrupciones; monitor de int_o.*	*cg_int.cp_irq_source × cp_ier_enable, bins RDA y TIMEOUT.*	*+UART_IER_BIT_EN_PCT*
V-IER-02	*F-IER-02*	*IER [1] controla la interrupción THRE. Se comprueba su generación cuando está habilitada y su enmascaramiento cuando está deshabilitada.*	*RAL; scoreboard; observación de IIR y int_o.*	*cg_int.cp_thre_event × cp_ier_enable*	*+UART_IER_BIT_EN_PCT*
V-IER-03	*F-IER-03*	*IER [2] habilita o enmascara la interrupción RLS ante errores de recepción.*	*RAL; scoreboard; información del agente serie.*	*cg_int.cp_rx_error × cp_ier_rls*	*+UART_IER_BIT_EN_PCT, +UART_SER_PARITY_ERR_PCT*
V-IER-04	*F-IER-04*	*IER [3] habilita o enmascara la interrupción de estado del módem ante cambios de CTS, DSR, RI o DCD.*	*RAL; monitor de módem; scoreboard de interrupciones.*	*cg_int.cp_modem_event × cp_ier_ms*	*+UART_IER_BIT_EN_PCT, +UART_MODEM_TOGGLE_PCT*
V-IER-05	*F-IER-05*	*Después del reset, IER vale 0x00. Se comprueba que los*	*RAL; comprobación de reset y lecturas*	*cg_reg.cp_ier_reset; cg_reg.cp_ier_reserved*	*Ninguno específico; configuración base.*

ID ítem	Característica	Qué se verifica	Mecanismo (scoreboard, SVA, RAL, cobertura)	Métrica de cierre (coverpoint o aserción)	Plusargs que lo dirigen
		bits reservados [7:4] se lean como cero.	*Wishbone.*		
V-IO-01	*F-IO-01*	*int_o se activa ante interrupciones habilitadas y pendientes. Se desactiva cuando no queda ninguna interrupción pendiente.*	*Scoreboard de interrupciones; monitor de int_o; RAL para consultar IIR.*	*cg_int.cp_irq_state × cp_pending*	*+UART_IER_BIT_EN_PCT, +UART_SW_SERVICE_DELAY_MAX*
V-IO-02	*F-IO-02*	*Con la salida opcional baud_o presente, su periodo es de DL ciclos de reloj (16 × baudios) con el divisor configurado*	*Monitor de baud_o; SVA (periodo contra regs.dl por puerta trasera)*	*cg_line_cfg.cp_dl; cover de la aserción del periodo de baud_o*	*+UART_LCR_DL_MIN, +UART_LCR_DL_MAX_FAST, +UART_LCR_DL_MAX, +UART_LCR_DL_TAIL_PCT*
V-IO-03	*F-IO-03*	*stx_pad_o permanece en 1 cuando no se transmite; las tramas que salen por stx_pad_o y las que entran por srx_pad_i respetan el formato de LCR y la tasa de DL*	*Agente serie (driver y monitor); scoreboard TX y RX; SVA de STX*	*cg_line_cfg.cp_wls × cp_stb × cp_dir*	*+UART_LCR_WLS_WEIGHTS, +UART_LCR_STB_PCT, +UART_LCR_PARITY_WEIGHTS*
V-IO-04	*F-IO-04*	*Las entradas CTS, DSR, RI y DCD pueden cambiar de estado. Las salidas RTS y DTR responden a la configuración del MCR, respetando la interpretación de polaridad documentada.*	*Driver y monitors de módem; scoreboard; RAL.*	*cg_modem.cp_signal × cp_level*	*+UART_MODEM_TOGGLE_PCT, +UART_MCR_CTRL_ONE_PCT*
V-IO-05	*F-IO-05*	*int_o refleja únicamente las interrupciones habilitadas y pendientes. Se comprueba que una fuente enmascarada no active la salida y que una fuente habilitada sí pueda activarla.*	*Scoreboard de interrupciones; monitor de int_o; RAL para configurar IER y observar IIR.*	*cg_int.cp_ier_mask × cp_irq_state*	*+UART_IER_BIT_EN_PCT*
V-INT-01	*F-INT-01*	*IIR [0] vale 0 cuando existe una interrupción pendiente y 1 cuando no existe ninguna.*	*RAL; scoreboard de interrupciones.*	*cg_int.cp_iir_pending × cp_irq_state*	*+UART_IER_BIT_EN_PCT*
V-INT-02	*F-INT-02*	*Ante fuentes simultáneas, IIR identifica la interrupción de mayor prioridad: RLS, RDA/timeout, THRE y Modem Status, según la*	*Modelo independiente de prioridades; scoreboard; monitor de int_o.*	*cg_int.cp_priority_pair × cp_iir_id*	*+UART_CFG_IRQ_OVERLAP_PCT, +UART_IER_BIT_EN_PCT*

ID ítem	Característica	Qué se verifica	Mecanismo (scoreboard, SVA, RAL, cobertura)	Métrica de cierre (coverpoint o aserción)	Plusargs que lo dirigen
		prioridad especificada.			
V-INT-03	*F-INT-03*	*Los errores parity, overrun, framing y break generan una interrupción RLS cuando IER [2] está habilitado. Se verifica su identificación y limpieza mediante lectura de LSR.*	*Scoreboard; RAL; monitor serie; información del modelo de errores.*	*cg_int.cp_rx_error × cp_iir_id, bin RLS.*	*+UART_IER_BIT_EN_PCT*
V-INT-04	*F-INT-04*	*La interrupción RDA se genera cuando la FIFO RX alcanza el nivel de disparo configurado: 1, 4, 8 o 14 caracteres.*	*Scoreboard de interrupciones; modelo de ocupación FIFO; RAL.*	*cg_int.cp_fifo_trigger × cp_iir_id, bin RDA.*	*+UART_FCR_TL_WEIGHTS, +UART_SW_SERVICE_DELAY_MAX*
V-INT-05	*F-INT-05*	*Se genera una interrupción de timeout cuando existen datos pendientes en RX, sin alcanzar el nivel de disparo, y transcurren cuatro tiempos de carácter sin actividad relevante.*	*Modelo temporal de interrupciones; scoreboard; monitor de tráfico RX.*	*cg_int.cp_iir_id, bin TIMEOUT, cruzado con cp_fifo_trigger.*	*+UART_SW_SERVICE_DELAY_MIN, +UART_SW_SERVICE_DELAY_MAX*
V-INT-06	*F-INT-06*	*La condición THRE genera su interrupción cuando está habilitada. Se comprueban los mecanismos de limpieza mediante lectura de IIR y escritura en THR.*	*Scoreboard; RAL; monitor Wishbone.*	*cg_int.cp_iir_id × cp_service_action, bin THRE.*	*+UART_IER_BIT_EN_PCT, +UART_SW_SERVICE_MODE_WEIGHTS*
V-INT-07	*F-INT-07*	*Las variaciones de las señales CTS, DSR, RI y DCD generan interrupciones de Modem Status cuando están habilitadas. La lectura de MSR permite atender el evento.*	*Monitor de módem; scoreboard; RAL.*	*cg_int.cp_modem_source × cp_iir_id, bin MODEM.*	*+UART_MODEM_TOGGLE_PCT, +UART_IER_BIT_EN_PCT*
V-INT-08	*F-INT-08*	*Después del reset, IIR vale 0xC1. Durante su funcionamiento, los bits [7:6] permanecen en 1 y [5:4] en 0.*	*RAL; comprobación de reset; scoreboard.*	*cg_int.cp_iir_reset; cg_int.cp_iir_fixed_bits*	*Ninguno específico; configurac ión base.*
V-INT-09	*F-INT-09*	*Una interrupción RLS pendiente se limpia al leer LSR. Se comprueba que IIR deje de identificar RLS y que int_o responda según las*	*Scoreboard de interrupciones; RAL; monitor Wishbone; monitor de int_o.*	*cg_int.cp_iir_id × cp_service_action × cp_pending_after, bin RLS y READ_LSR.*	*+UART_IER_BIT_EN_PCT, +UART_SW_SERVICE_MODE_WEIGHTS*

ID ítem	Característica	Qué se verifica	Mecanismo (scoreboard, SVA, RAL, cobertura)	Métrica de cierre (coverpoint o aserción)	Plusargs que lo dirigen
		interrupciones que permanezcan pendientes.			
V-INT-10	*F-INT-10*	*La interrupción RDA se limpia cuando la cantidad de caracteres en FIFO RX disminuye por debajo del nivel de disparo configurado mediante lecturas de RBR.*	*Scoreboard; modelo de ocupación FIFO; RAL; monitor Wishbone.*	*cg_int.cp_fifo_trigger × cp_fifo_below_trigger × cp_pending_after*	*+UART_FCR_TL_WEIGHTS, +UART_SW_SERVICE_DELAY_MAX*
V-INT-11	*F-INT-11*	*Una interrupción de timeout pendiente se limpia mediante una lectura de RBR. Se comprueba que IIR deje de identificar timeout y que las demás fuentes pendientes continúen siendo atendidas.*	*Modelo temporal de interrupciones; scoreboard; RAL; monitor Wishbone.*	*cg_int.cp_iir_id × cp_service_action × cp_pending_after, bin TIMEOUT y READ_RBR.*	*+UART_SW_SERVICE_DELAY_MIN, +UART_SW_SERVICE_DELAY_MAX*
V-INT-12	*F-INT-12*	*La interrupción THRE se limpia mediante una escritura en THR o mediante una lectura de IIR cuando THRE es la interrupción identificada. Se comprueban ambos mecanismos.*	*Scoreboard de interrupciones; RAL; monitor Wishbone.*	*cg_int.cp_iir_id × cp_service_action × cp_pending_after, bin THRE con WRITE_THR y READ_IIR.*	*+UART_IER_BIT_EN_PCT, +UART_SW_SERVICE_MODE_WEIGHTS*
V-INT-13	*F-INT-13*	*Una interrupción de Modem Status pendiente se limpia al leer MSR. Se comprueba la limpieza de los indicadores de cambio y que int_o se actualice según las fuentes pendientes restantes.*	*Monitor de módem; scoreboard de interrupciones; RAL; monitor Wishbone.*	*cg_int.cp_iir_id × cp_service_action × cp_pending_after, bin MODEM y READ_MSR.*	*+UART_MODEM_TOGGLE_PCT, +UART_SW_SERVICE_DELAY_MAX*
V-INT-14	*F-INT-14*	*Cuando coinciden varias fuentes de interrupción, se comprueba que IIR identifique la de mayor prioridad: RLS (4'h6), RDA (4'h4) o timeout (4'hC), THRE (4'h2) y Modem Status (4'h0). Se verifican las combinaciones simultáneas alcanzables.*	*Modelo independiente de prioridades; scoreboard de interrupciones; monitor de int_o; RAL para observar IIR.*	*cg_int.cp_priority_pair × cp_iir_id*	*+UART_CFG_IRQ_OVERLAP_PCT, +UART_IER_BIT_EN_PCT*
V-MCR-01	*F-MCR-01*	*MCR [0] controla DTR. Se comprueban las configuraciones 0 y*	*RAL; monitor de dtr_pad_o; scoreboard.*	*cg_modem.cp_dtr_cfg × cp_dtr_pin*	*+UART_MCR_CTRL_ONE_PCT*

ID ítem	Característica	Qué se verifica	Mecanismo (scoreboard, SVA, RAL, cobertura)	Métrica de cierre (coverpoint o aserción)	Plusargs que lo dirigen
		1 y la polaridad esperada de la salida física.			
V-MCR-02	*F-MCR-02*	*MCR [1] controla RTS. Se comprueban las configuraciones 0 y 1 y la polaridad esperada de la salida física.*	*RAL; monitor de rts_pad_o; scoreboard.*	*cg_modem.cp_rts_cfg × cp_rts_pin*	*+UART_MCR_CTRL_ONE_PCT*
V-MCR-03	*F-MCR-03*	*En modo loopback, MCR [2] (Out1) se refleja en el estado lógico RI correspondiente, observable mediante MSR.*	*RAL; scoreboard de módem; lectura de MSR.*	*cg_modem.cp_loopback × cp_out1*	*+UART_MCR_CTRL_ONE_PCT, +UART_MCR_LOOPBACK_PCT*
V-MCR-04	*F-MCR-04*	*En modo loopback, MCR [3] (Out2) se refleja en el estado lógico DCD correspondiente, observable mediante MSR.*	*RAL; scoreboard de módem; lectura de MSR.*	*cg_modem.cp_loopback × cp_out2*	*+UART_MCR_CTRL_ONE_PCT, +UART_MCR_LOOPBACK_PCT*
V-MCR-05	*F-MCR-05*	*Con MCR [4] activo, stx_pad_o permanece en 1 y la ruta interna del transmisor se conecta al receptor. Se comprueba que el tráfico interno se reciba correctamente.*	*SVA de STX; scoreboard serie compartido con Diego; RAL.*	*cover property de loopback activo; cg_modem.cp_loopback × cp_serial_activity*	*+UART_MCR_LOOPBACK_PCT*
V-MCR-06	*F-MCR-06*	*En loopback se comprueba la correspondencia DTR→DSR, RTS→CTS, Out1→RI y Out2→DCD, utilizando la representación lógica definida por la especificación.*	*Scoreboard de módem; monitor; RAL; aserciones SVA.*	*cg_modem.cp_loopback × cp_modem_signal*	*+UART_MCR_LOOPBACK_PCT, +UART_MCR_CTRL_ONE_PCT*
V-MCR-07	*F-MCR-07*	*MCR se comporta como registro de escritura únicamente. Los bits [7:5] se ignoran y el registro se reinicia a cero. No se exige recuperar su contenido mediante lectura normal.*	*RAL; comprobación de reset; observación de efectos y lectura Debug opcional.*	*cg_modem.cp_mcr_cfg; cg_modem.cp_mcr_reset*	*+UART_MCR_CTRL_ONE_PCT*
V-MSR-01	*F-MSR-01*	*MSR [0] (DCTS) se activa cuando CTS cambia de estado. Se comprueban transiciones ascendentes y descendentes.*	*Monitor de CTS; scoreboard; lectura de MSR mediante RAL.*	*cg_modem.cp_cts_edge × cp_dcts*	*+UART_MODEM_TOGGLE_PCT*

ID ítem	Característica	Qué se verifica	Mecanismo (scoreboard, SVA, RAL, cobertura)	Métrica de cierre (coverpoint o aserción)	Plusargs que lo dirigen
V-MSR-02	*F-MSR-02*	*MSR [1] (DDSR) se activa cuando DSR cambia de estado. Se comprueban transiciones ascendentes y descendentes.*	*Monitor de DSR; scoreboard; RAL.*	*cg_modem.cp_dsr_edge × cp_ddsr*	*+UART_MODEM_TOGGLE_PCT*
V-MSR-03	*F-MSR-03*	*MSR [2] (TERI) se activa ante la transición de RI especificada. Se comprueban ambos sentidos de cambio para detectar la discrepancia AMB-04.*	*Monitor de RI; scoreboard; lectura de MSR mediante RAL.*	*cg_modem.cp_ri_edge × cp_teri*	*+UART_MODEM_TOGGLE_PCT*
V-MSR-04	*F-MSR-04*	*MSR [3] (DDCD) se activa cuando DCD cambia de estado. Se comprueban transiciones ascendentes y descendentes.*	*Monitor de DCD; scoreboard; RAL.*	*cg_modem.cp_dcd_edge × cp_ddcd*	*+UART_MODEM_TOGGLE_PCT*
V-MSR-05	*F-MSR-05*	*Los indicadores MSR [3:0] se mantienen ante los eventos correspondientes y se limpian al leer MSR. Se comprueba la limpieza sin alterar los bits de estado actual.*	*RAL; monitor Wishbone; scoreboard de módem.*	*cg_modem.cp_delta_type × cp_clear_event*	*+UART_MODEM_TOGGLE_PCT, +UART_SW_SERVICE_DELAY_MAX*
V-MSR-06	*F-MSR-06*	*MSR [7:4] refleja el estado de las entradas de módem según la polaridad especificada. En loopback refleja las señales internas asociadas a MCR.*	*Scoreboard de módem; RAL; monitores de pines.*	*cg_modem.cp_loopback × cp_signal × cp_level*	*+UART_MODEM_TOGGLE_PCT, +UART_MCR_LOOPBACK_PCT*
V-TX-01	*F-TX-01*	*Cada byte escrito en THR (dirección 0 con DLAB = 0) entra a la FIFO TX y sale por stx_pad_o en el orden de escritura, con la longitud, la paridad y los bits de parada de LCR y la tasa de DL*	*Scoreboard TX (modelo de FIFO TX); monitor TX; RAL*	*cg_tx.cp_tx_fifo_level × cg_line_cfg.cp_wls*	*+UART_LCR_WLS_WEIGHTS, +UART_LCR_PARITY_WEIGHTS, +UART_LCR_STB_PCT, +UART_TX_BURST_MAX*
V-TX-02	*F-TX-02*	*La FIFO TX almacena 16 caracteres: con 16 escrituras seguidas a THR se transmiten completos y en orden, y una escritura con la FIFO llena no altera los caracteres almacenados*	*Scoreboard TX (modelo de FIFO de 16 posiciones); Debug 2 (tf_count)*	*cg_tx.cp_tx_fifo_level[16] × cp_overflow*	*+UART_TX_BURST_MAX, +UART_TX_OVERFLOW_PCT*

ID ítem	Característica	Qué se verifica	Mecanismo (scoreboard, SVA, RAL, cobertura)	Métrica de cierre (coverpoint o aserción)	Plusargs que lo dirigen
V-RX-01	*F-RX-01*	*Los caracteres recibidos por srx_pad_i entran a la FIFO RX y salen en orden por RBR (dirección 0 con DLAB = 0); cada lectura de RBR extrae un carácter*	*Scoreboard RX (modelo de FIFO RX); monitor serie; RAL*	*cg_rx.cp_rx_fifo_level[0:16]* × *cg_line_cfg.cp_wls*	*+UART_LCR_WLS_WEIGHTS, +UART_SW_SERVICE_DELAY_MAX*

6. Estrategia de estímulo: prueba base

6.1 Espacio de aleatorización

Variable aleatoria	Objeto de configuración	Rango legal	Distribución por defecto	Knobs
dl	*uart_line_cfg*	*1 a 65535*	*98 % en 1 a 16, 2 % en el resto*	*dl_max_fast, dl_max, dl_tail_pct*
wb_delay_cycles	*wb_agent_cfg*	*0 a 255 ciclos de reloj de reposo entre transacciones*	*98 % uniforme entre 0 y 8 ciclos; 2 % uniforme entre 9 y 255 ciclos.*	*wb_delay_fast_max, wb_delay_max, wb_delay_tail_pct*
wb_stb_delay_cycles	*wb_agent_cfg*	*0 a 255 ciclos entre la subida de cyc y la de stb*	*Uniforme entre 0 y 4*	*wb_stb_delay_max*
wb_sel	*wb_agent_cfg*	*4'b0001, 4'b0010, 4'b0100 o 4'b1000 en registros normales; 4'b1111 en Debug 1 y 2*	*Pesos 1, 1, 1, 1 entre las cuatro posiciones; 4'b1111 en todos los accesos a Debug*	*wb_sel_w[4]*
wb_sel_illegal	*wb_agent_cfg*	*0 o 1 por acceso (0 bits o 2 o más bits activos en wb_sel_i)*	*0 % (caso no legal: la especificación no lo define)*	*wb_sel_illegal_pct*
reset_request	*uart_env_cfg*	*0 o 1 por sesión de tráfico*	*Apagado (rst_en = 0); con rst_en = 1, 10 % de las sesiones incluyen un reset a media sesión*	*rst_en, rst_mid_traffic_pct*
reset_phase	*uart_env_cfg*	*Reposo, TX activo, RX activo, FIFO llena, DLAB = 1*	*Uniforme entre las cinco fases cuando hay reset*	*—*
lcr.wls	*uart_line_cfg*	*0 a 3*	*Uniforme*	*wls_w[4]*
lcr.stb	*uart_line_cfg*	*0 o 1*	*50 % / 50 %*	*stb_pct*
Tipo de paridad (lcr.pen, lcr.eps, lcr.sp)	*uart_line_cfg*	*NONE, ODD, EVEN, STICK1, STICK0*	*Pesos 2, 1, 1, 1, 1*	*parity_w[5]*
lcr.eps y lcr.sp con lcr.pen = 0	*uart_line_cfg*	*0 o 1*	*Uniforme (no afectan la trama; se varían para la cobertura de toggle)*	*—*
lcr.bc	*uart_line_cfg*	*0 o 1*	*5 % de probabilidad de activar el break por sesión de tráfico*	*bc_pct*
bc_len_chars	*uart_line_cfg*	*1 a 4 tiempos de carácter*	*Uniforme entre 1 y 2; el resto con un plusarg*	*bc_len_max*
dl	*uart_line_cfg*	*0 a 65535*	*1 % en 0 (solo en ventanas cortas de reconfiguración), 97 % en 1 a 16, 2 % en 17 a 65535*	*dl_min, dl_max_fast, dl_max, dl_tail_pct, dl_zero_pct*
dl_write_order	*uart_line_cfg*	*DLM→DLL, solo DLL, DLL→DLM*	*Pesos 8, 1, 1*	*dl_order_w[3]*
fcr.tl	*uart_fifo_cfg*	*0 a 3 (1, 4, 8 o 14 bytes)*	*Uniforme*	*tl_w[4]*
fcr.b0 y fcr.b5_3	*uart_fifo_cfg*	*b0: 0 o 1; b5_3: 0 a 7*	*En el 50 % de las escrituras a FCR se ponen valores aleatorios; en el resto, en 0*	*fcr_ign_bits_pct*
fcr_clr_rx y fcr_clr_tx	*uart_fifo_cfg*	*0 o 1 cada uno*	*3 % de las escrituras a FCR limpian cada FIFO*	*fcr_clr_pct*
reg_addr_class	*uart_env_cfg*	*Registros normales,*	*Pesos 8, 1, 1 en los*	*reg_addr_w[3]*

Variable aleatoria	Objeto de configuración	Rango legal	Distribución por defecto	Knobs
		Debug, direcciones no mapeadas	*accesos aleatorios a registros*	
reg_dlab_access	*uart_env_cfg*	*0 o 1*	*10 % de los accesos aleatorios se hacen con DLAB = 1*	*reg_dlab_pct*
tx_burst_len	*uart_env_cfg*	*1 a 20 escrituras a THR*	*95 % uniforme entre 1 y 16; 5 % entre 17 y 20 (desborda la FIFO TX)*	*tx_burst_min, tx_burst_max, tx_overflow_pct*
ser_gap_bits	*uart_ser_agent_cfg*	*0 a 255 tiempos de bit entre caracteres RX*	*Uniforme entre 0 y 20*	*ser_gap_max*
ser_parity_err	*uart_ser_agent_cfg*	*0 o 1 por carácter (solo con paridad habilitada)*	*5 % de los caracteres*	*ser_parity_err_pct*
ser_framing_err	*uart_ser_agent_cfg*	*0 o 1 por carácter*	*5 % de los caracteres*	*ser_framing_err_pct*
ser_stop2_err	*uart_ser_agent_cfg*	*0 o 1 por carácter (solo con 1.5 o 2 bits de parada)*	*5 % de los caracteres*	*ser_stop2_err_pct*
ser_break	*uart_ser_agent_cfg*	*0 o 1 por hueco entre caracteres*	*3 % de los huecos*	*ser_break_pct*
ser_break_len_chars	*uart_ser_agent_cfg*	*1 a 8 tiempos de carácter*	*Uniforme entre 1 y 3*	*ser_break_len_max*
ser_glitch	*uart_ser_agent_cfg*	*0 o 1 por hueco entre caracteres; ancho de 1 a 7 ticks (de 16 por bit)*	*3 % de los huecos*	*ser_glitch_pct*
ser_baud_dev	*uart_ser_agent_cfg*	*−10 % a +10 % del periodo de bit*	*90 % sin desviación; 10 % uniforme en todo el rango*	*ser_baud_dev_pct, ser_baud_dev_max*
modem_toggle_mask[3:0]	*modem_agent_cfg*	*0000 a 1111, una posición por CTS, DSR, RI y DCD*	*Cada entrada tiene 20 % de probabilidad de cambiar por oportunidad de estímulo*	*modem_toggle_pct*
mcr_ctrl[3:0]	*modem_agent_cfg*	*0000 a 1111, correspondientes a DTR, RTS, Out1 y Out2*	*Cada bit tiene 50 % de probabilidad de valer 1*	*mcr_ctrl_one_pct*
mcr_loopback	*modem_agent_cfg*	*0 o 1*	*15 % loopback y 85 % modo normal*	*mcr_loopback_pct*
ier_bits[3:0]	*uart_env_cfg*	*0000 a 1111, las 16 máscaras posibles*	*Cada bit tiene 50 % de probabilidad de estar habilitado*	*ier_bit_en_pct*
irq_overlap_request	*uart_env_cfg*	*0 o 1*	*10 % de probabilidad de programar coincidencia de interrupciones*	*cfg_irq_overlap_pct*
reconfig_request	*uart_env_cfg*	*0 o 1*	*10 % de probabilidad de solicitar una reconfiguración durante la sesión*	*cfg_reconfig_pct*
service_mode	*uart_env_cfg*	*0: sondeo, 1: servicio por interrupción*	*Pesos 1,1. Ambos modos tienen igual probabilidad*	*sw_service_mode_weights[2]*
service_delay_chars	*uart_env_cfg*	*0 a 255 tiempos de carácter, como rango inicial del generador*	*98 % entre 0 y 20, 2 % entre 21 y 255*	*sw_service_delay_min, sw_service_delay_fast_max, sw_service_delay_max, sw_service_delay_tail*

6.2 Argumento de completitud

Bin o grupo de bins	Por qué su probabilidad es mayor que cero con los valores por defecto	Plusarg que lo acelera
cg_err.cp_oe × cp_tl	*La FIFO RX se llena si el retraso de servicio aleatorio supera el tiempo de 16 caracteres; el rango por defecto del retraso lo permite*	*+UART_SW_SERVICE_DELAY_MIN*
cg_bus.cp_sel × cp_dir	*Cada acceso a un registro normal elige uno de los cuatro bytes con pesos 1, 1, 1, 1, así que cada bin sale en ≈ 25 % de esos accesos, en lectura y en escritura; los accesos a Debug usan 4'b1111*	*+UART_WB_SEL_WEIGHTS*
cg_bus.cp_sel_illegal [0 bits, 2 o más bits]	*No es un escenario legal: la especificación no define el comportamiento con 0 o con más de un bit activo en wb_sel_i, por eso su probabilidad por defecto es 0 y solo se ejerce con el plusarg; el bin se lista en el informe de cierre*	*+UART_WB_SEL_ILLEGAL_PCT*
cg_bus.cp_ack_latency	*Con retardos uniformes de 0 a 8 ciclos entre transacciones y de 0 a 4 ciclos entre cyc y stb, el monitor observa accesos consecutivos y separados, y mide la latencia de ack en cada ciclo cyc && stb. Los límites 8 y 4 se justifican porque un retardo mayor no ejerce lógica nueva de la interfaz Wishbone: el efecto del tiempo entre accesos sobre los eventos del UART (timeout, overrun) lo cubre el retraso de servicio*	*+UART_WB_DELAY_MAX, +UART_WB_STB_DELAY_MAX*
cg_rst.cp_reset_phase	*El reset aleatorio está apagado por defecto: un reset a media sesión trunca los escenarios largos (overrun, timeout, ráfagas que llenan la FIFO TX) y reduce su cobertura. El reset inicial siempre ocurre y cubre la fase de reposo. Con +UART_RST_EN=1, el 10 % de las sesiones incluye un reset en una fase uniforme entre reposo, TX activo, RX activo, FIFO llena y DLAB = 1. Los bins TX activo, RX activo, FIFO llena y DLAB = 1 se listan en el informe de cierre porque solo se alcanzan con plusargs*	*+UART_RST_EN, +UART_RST_MID_TRAFFIC_PCT*
cg_regs.cp_addr [7 (scratch)] × cp_dir	*El registro Scratch de la dirección interna 7 se incluye entre los registros implementados por el RTL. La selección aleatoria de direcciones permite generar accesos de lectura y escritura a este registro con probabilidad positiva mediante los pesos predeterminados de +UART_REG_ADDR_WEIGHTS. Se documenta como una extensión de la implementación y no como una funcionalidad exigida por la especificación.*	*+UART_REG_ADDR_WEIGHTS*
cg_regs.cp_addr [8, 12] × cp_dir × cp_dbg_core_state	*El 10 % de los accesos aleatorios va a Debug (pesos 8, 1, 1) con wb_sel_i = 4'b1111. El estado del núcleo en ese momento (FIFOs con datos, TX o RX en curso, bits pendientes en LSR, MSR e IIR) lo fijan el tráfico serie con errores, los cambios de módem y los*	*+UART_REG_ADDR_WEIGHTS, +UART_SW_SERVICE_DELAY_MAX*

Bin o grupo de bins	Por qué su probabilidad es mayor que cero con los valores por defecto	Plusarg que lo acelera
	retrasos de servicio, así que Debug 1 y 2 se leen con el núcleo en reposo y con estado pendiente	
cg_regs.cp_addr[0:6] (variante de 8 bits)	*Solo se alcanza al compilar con +define+DATA_BUS_WIDTH_8: el modo de 32 bits es el obligatorio y el de 8 bits (puntos extra) corre en una regresión aparte. El bin se lista en el informe de cierre porque solo se alcanza con el define y el plusarg*	*+UART_WB_BUS_WIDTH*
cg_rx.cp_rx_fifo_level [0 a 16] × cg_line_cfg.cp_wls	*El nivel sube con los caracteres RX y baja con las lecturas de RBR. El retraso de servicio (98 % entre 0 y 20 tiempos de carácter, 2 % entre 21 y 255) deja la FIFO en niveles bajos con los retrasos cortos y la llena si supera el tiempo de 16 caracteres; cada longitud de carácter tiene la misma probabilidad (1/4)*	*+UART_SW_SERVICE_DELAY_MAX, +UART_SW_SERVICE_DELAY_TAIL_PCT, +UART_LCR_WLS_WEIGHTS*
cg_tx.cp_tx_fifo_level [0 a 15] × cp_tx_busy	*El 95 % de las ráfagas tiene entre 1 y 16 escrituras a THR, que llevan la FIFO TX por niveles intermedios; al vaciarse, el nivel 0 ocurre con el registro de desplazamiento ocupado y libre, así que THRE y TEMT toman valores distintos*	*+UART_TX_BURST_MIN, +UART_TX_BURST_MAX*
cg_err.cp_lsr_b7 × cp_error_kind [PE, FE, BI]	*LSR[7] depende de los errores de paridad, de trama y de break dentro de la FIFO: los de paridad (≈ 3,3 % de los caracteres RX) y los de trama (5 %) producen algún error en ≈ 8 % de los caracteres, y el break aparece en el 3 % de los huecos entre caracteres*	*+UART_SER_PARITY_ERR_PCT, +UART_SER_FRAMING_ERR_PCT, +UART_SER_BREAK_PCT*
cg_line_cfg.cp_stb × cp_wls [1.5 bits de parada]	*El caso de 1.5 bits ocurre con WLS = 5 bits y STB = 1. Con WLS uniforme (1/4) y STB al 50 %, aparece en el 12,5 % de las configuraciones.*	*+UART_LCR_WLS_WEIGHTS, +UART_LCR_STB_PCT*
cg_line_cfg.cp_parity[STICK0, STICK1] × cp_dir	*Con pesos 2, 1, 1, 1, 1, cada tipo de paridad stick sale en 1/6 (≈ 17 %) de las configuraciones, tanto en TX como en RX.*	*+UART_LCR_PARITY_WEIGHTS*
cg_line_cfg.cp_dl[ZERO]	*DL = 0 tiene un 1 % de probabilidad por defecto, dentro de una ventana corta de reconfiguración, para no detener toda la prueba.*	*+UART_LCR_DL_ZERO_PCT*
cg_line_cfg.cp_dl [17 a 255, 256 a 65534, 65535]	*La cola tiene un 2 % repartido en partes iguales entre los tres bins, así que cada uno sale en ≈ 0,67 % de las configuraciones. Por el tiempo de simulación, la cola se acota a ese porcentaje.*	*+UART_LCR_DL_TAIL_PCT, +UART_LCR_DL_MAX*
cg_line_cfg.cp_bc	*El break en TX se activa en el 5 % de las sesiones de tráfico, con una duración aleatoria de 1 a 2 tiempos de carácter.*	*+UART_LCR_BC_PCT, +UART_LCR_BC_LEN_MAX*
cg_reconfig.cp_dl_order y cg_regs.cp_init_seq	*En cada configuración (inicial o reconfiguración), los órdenes alternativos "solo DLL" y "DLL→DLM" tienen un 10 % de probabilidad cada uno.*	*+UART_LCR_DL_ORDER_WEIGHTS, +UART_CFG_RECONFIG_PCT*
cg_fifo_cfg.cp_rx_clr × cp_rx_busy y cp_tx_clr × cp_tx_busy	*Limpiar con un carácter en curso requiere escribir FCR durante el*	*+UART_FCR_CLR_PCT, +UART_CFG_RECONFIG_PCT*

Bin o grupo de bins	Por qué su probabilidad es mayor que cero con los valores por defecto	Plusarg que lo acelera
	tráfico: la reconfiguración en medio del tráfico (10 %) combinada con la limpieza (3 % de las escrituras a FCR) lo hace posible.	
cg_fifo_cfg.cp_fcr_b0 y cp_fcr_b5_3	*En el 50 % de las escrituras a FCR, los bits ignorados toman valores aleatorios, así que se alcanzan todos sus valores.*	*+UART_FCR_IGN_BITS_PCT*
cg_tx.cp_tx_fifo_level[16] × cp_overflow	*El 5 % de las ráfagas tiene entre 17 y 20 escrituras, lo que llena la FIFO TX y escribe con la FIFO llena.*	*+UART_TX_OVERFLOW_PCT, +UART_TX_BURST_MAX*
cg_regs.cp_addr [8, 12, no mapeadas]	*Con pesos 8, 1, 1, el 10 % de los accesos aleatorios va a Debug y otro 10 % a direcciones no mapeadas.*	*+UART_REG_ADDR_WEIGHTS*
cg_regs.cp_addr × cp_dir × cp_dlab	*El 10 % de los accesos aleatorios se hace con DLAB = 1, así que se alcanzan las lecturas y escrituras de DLL y DLM fuera de la secuencia de configuración.*	*+UART_REG_DLAB_PCT*
cg_err.cp_stop2_err	*Requiere STB = 1 (50 %) y que el driver ponga un 0 en el segundo bit de parada (5 %): ocurre en el 2,5 % de los caracteres RX.*	*+UART_SER_STOP2_ERR_PCT, +UART_LCR_STB_PCT*
cg_err.cp_pe × cg_line_cfg.cp_parity	*La paridad está habilitada en 4 de 6 partes de las configuraciones y el 5 % de esos caracteres lleva paridad corrupta: ≈ 3,3 % de los caracteres RX.*	*+UART_SER_PARITY_ERR_PCT*
cg_err.cp_fe × cg_line_cfg.cp_wls	*El 5 % de los caracteres RX lleva el primer bit de parada en 0, con cualquier longitud de carácter.*	*+UART_SER_FRAMING_ERR_PCT*
cg_err.cp_bi × cp_break_len	*El 3 % de los huecos entre caracteres RX se convierte en un break de 1 a 3 tiempos de carácter, más largo que el tiempo mínimo de detección.*	*+UART_SER_BREAK_PCT, +UART_SER_BREAK_LEN_MAX*
cg_ser.cp_glitch	*El 3 % de los huecos entre caracteres RX lleva un pulso en 0 de 1 a 7 ticks, siempre más corto que medio bit.*	*+UART_SER_GLITCH_PCT*
cg_ser.cp_baud_dev [dentro y fuera de tolerancia]	*El 10 % de los caracteres lleva una desviación uniforme entre −10 % y +10 %: ≈ 3 % cae dentro de ±3 % (debe recibirse bien) y ≈ 4 % cae fuera de ±6 % (se espera error).*	*+UART_SER_BAUD_DEV_PCT, +UART_SER_BAUD_DEV_MAX*
cg_modem.cp_signal × cp_transition × cp_delta	*Cada entrada CTS, DSR, RI y DCD tiene un 20 % de probabilidad de cambiar en cada oportunidad de estímulo. La secuencia permite mantener o modificar cada señal y generar transiciones ascendentes y descendentes. Esto permite alcanzar los eventos DCTS, DDSR, TERI y DDCD.*	*+UART_MODEM_TOGGLE_PCT*
cg_modem.cp_loopback × cp_modem_signal × cp_level	*El modo loopback tiene una probabilidad predeterminada del 15 % y cada bit de control MCR [3:0] tiene un 50 % de probabilidad de valer 1. Por tanto, pueden generarse las configuraciones legales de control de módem tanto en modo normal como en loopback.*	*+UART_MCR_LOOPBACK_PCT, +UART_MCR_CTRL_ONE_PCT*
cg_int.cp_ier_mask × cp_irq_source ×	*Cada bit IER [3:0] tiene un 50 % de*	*+UART_IER_BIT_EN_PCT*

Bin o grupo de bins	Por qué su probabilidad es mayor que cero con los valores por defecto	Plusarg que lo acelera
cp_irq_state	*probabilidad de estar habilitado, por lo que las 16 máscaras posibles son alcanzables. La secuencia virtual combina estas máscaras con tráfico serie y eventos de módem para producir fuentes habilitadas y enmascaradas.*	
cg_int.cp_priority_pair × cp_iir_id	*Existe un 10 % de probabilidad de solicitar escenarios con interrupciones simultáneas. La secuencia virtual puede habilitar varias fuentes y generar sus condiciones antes de atenderlas, permitiendo comprobar las prioridades de IIR.*	*+UART_CFG_IRQ_OVERLAP_PCT, +UART_IER_BIT_EN_PCT*
cg_int.cp_fifo_trigger × cp_iir_id [RDA]	*Los niveles de disparo FIFO de 1, 4, 8 y 14 caracteres tienen pesos predeterminados positivos.La generación de tráfico RX y los retrasos de servicio permiten alcanzar cada nivel antes de retirar los caracteres.*	*+UART_FCR_TL_WEIGHTS, +UART_SW_SERVICE_DELAY_MAX*
cg_int.cp_fifo_trigger × cp_iir_id[TIMEOUT]	*La prueba puede generar datos en RX, mantener la ocupación por debajo del nivel de disparo y retrasar las lecturas durante al menos cuatro tiempos de carácter. La distribución predeterminada incluye retrasos largos.*	*+UART_SW_SERVICE_DELAY_TAIL_PCT, +UART_FCR_TL_WEIGHTS*
cg_int.cp_iir_id × cp_service_action × cp_pending_after	*Los modos de sondeo y servicio por interrupción tienen pesos predeterminados positivos. La secuencia virtual puede generar cada fuente de interrupción y ejecutar su mecanismo correspondiente de limpieza mediante accesos RAL. Los retrasos permiten observar interrupciones pendientes antes y después del servicio.*	*+UART_SW_SERVICE_MODE_WEIGHTS, +UART_SW_SERVICE_DELAY_MAX*
cg_int.cp_irq_state × cp_reconfig × cp_service_mode	*Existe un 10 % de probabilidad de solicitar una reconfiguración durante el tráfico y ambos modos de servicio tienen pesos positivos. La secuencia virtual permite modificar las configuraciones de interrupción y módem mientras existen operaciones en curso, respetando los accesos legales a registros.*	*+UART_CFG_RECONFIG_PCT, +UART_SW_SERVICE_MODE_WEIGHTS*
cg_bus.cp_inter_transaction_gap	*La distribución predeterminada genera retrasos entre transacciones de 0 a 255 ciclos, con una probabilidad del 98 % en la región de 0 a 8 y del 2 % en la región de 9 a 255. Por tanto, los intervalos cortos y largos tienen probabilidad positiva.*	*+UART_WB_DELAY_FAST_MAX, +UART_WB_DELAY_MAX, +UART_WB_DELAY_TAIL_PCT*

6.3 Secuencia virtual principal

```
SECUENCIA uart_base_vseq

  body():
    cfgs     = obtener cfg por el secuenciador           // E4
    n_rondas = sortear en [cfg. rounds_min : cfg. rounds_max]

    // Acto 1: reset
    aplicar_reset(duración sorteada)
    leer todos los registros y comprobar los valores de reset accesibles, sin efectos secundarios y verificar los registros
de solo escritura mediante sus efectos observables o en caso de ser necesario acceso backdoor para validar estado
actual.

    // Acto 2: configuración
    config = randomizar_config()       // DL, WLS, STB, paridad, FIFO en/TL, IER, MCR (con sus knobs)
    programar_uart(config)          // orden de driver: DLAB=1, DLL/DLM, DLAB=0, LCR, FCR, IER, MCR
    comprobar registros legibles y efectos observables

   // Los monitores actualizan el modelo de referencia

    // Acto 5: servicio de interrupciones, en segundo plano
    fork servicio_interrupciones () join_none

    // Actos 3 y 4: tráfico y reconfiguraciones
    PARA ronda = 1 .. n_rondas:
      mezcla = randomizar_mezcla()     // solo TX | solo RX | TX+RX | solo módem | todo fork
        SI mezcla incluye TX:     flujo_tx(n randomizado)
        SI mezcla incluye RX:     flujo_rx(n randomizado)
        SI mezcla incluye módem:  flujo_modem(n randomizado)
        SI sorteo < cfg.reconfig_pct:
          esperar un retardo randomizado dentro de la ronda   // línea ocupada o en reposo
          reconfigurar(randomizar_cambio())                   // DL | LCR | FCR | IER | MCR
      join
      SI cfg.rst_en Y randomizado < cfg.rst_mid_pct:
       // detener temporalmente servicio de interrupciones
      // registrar y cancelar operaciones afectadas
          aplicar_reset();
         limpiar_scoreboard();
         repetir Acto 2
    // reiniciar servicio de interrupciones


    // Acto 6: fin de prueba
    detener generadores
    esperar_reposo(máx. cfg.end_timeout tiempos de trama)   // línea en 1, FIFO TX vacía, TEMT=1
    drenar FIFO RX hasta LSR.DR == 0
    detener servicio_interrupciones
    verificar: int_o == 0, IIR sin pendientes, scoreboard sin esperados pendientes
    SI se agotó el tiempo: uvm_error("TIMEOUT")
```

7. Tabla de plusargs

Plusarg	Tipo y rango	Valor por defecto	Componentes que lo usan	Objeto de configuración	Efecto
+UART_WB_BUS_WIDTH	*Entero: 8 o 32*	*32*	*Driver Wishbone, adaptador reg2bus, scoreboard*	*wb_agent_cfg*	*Ancho del bus Wishbone y de la dirección (32 bits con 5 bits de dirección; 8 bits con 3 bits de dirección); se usa junto con +define+DATA_BUS_WIDTH_8*
+UART_WB_SEL_WEIGHTS	*Lista de 4 enteros ≥ 0 separados por comas, sin espacios (4'b0001, 4'b0010, 4'b0100, 4'b1000); al menos uno > 0*	*1,1,1,1*	*Driver Wishbone, adaptador reg2bus*	*wb_agent_cfg*	*Pesos del byte seleccionado con wb_sel_i en los accesos a registros normales*
+UART_WB_SEL_ILLEGAL_PCT	*Entero 0 a 100*	*0*	*Driver Wishbone, monitor Wishbone*	*wb_agent_cfg*	*% de accesos con wb_sel_i de cero bits o de dos o más bits activos (caso no definido por la especificación)*
+UART_WB_DELAY_FAST_MAX	*Entero de 0 a 254, menor que UART_WB_DELAY_MAX*	*8*	*Driver Wishbone*	*wb_agent_cfg*	*Define el límite superior de los retrasos frecuentes entre transacciones*
+UART_WB_DELAY_MAX	*Entero de 1 a 255, mayor que UART_WB_DELAY_FAST_MAX*	*255*	*Driver Wishbone*	*wb_agent_cfg*	*Define el máximo configurable del retraso entre transacciones, incluyendo la región larga*
+UART_WB_DELAY_TAIL_PCT	*Entero de 0 a 100*	*2*	*Driver Wishbone*	*wb_agent_cfg*	*Porcentaje de retrasos que se generan en la región comprendida entre FAST_MAX + 1 y MAX.*
+UART_WB_STB_DELAY_MAX	*Entero 0 a 255 (ciclos de reloj)*	*4*	*Driver Wishbone*	*wb_agent_cfg*	*Máximo de ciclos entre la subida de cyc y la de stb*
+UART_REG_ADDR_WEIGHTS	*Lista de 3 enteros ≥ 0 separados por comas, sin espacios (registros normales, Debug, direcciones no mapeadas); al menos uno > 0*	*8,1,1*	*Secuencia de registros, driver Wishbone*	*uart_env_cfg*	*Pesos de la clase de dirección en los accesos aleatorios a registros*
+UART_REG_DLAB_PCT	*Entero 0 a 100*	*10*	*Secuencia de registros*	*uart_env_cfg*	*% de accesos aleatorios a registros que se hacen con DLAB = 1*
+UART_RST_EN	*0 o 1*	*0*	*Secuencia virtual, scoreboard*	*uart_env_cfg*	*Habilita el reset aleatorio a media sesión*
+UART_RST_MID	*Entero 0 a 100*	*10*	*Secuencia virtual,*	*uart_env_cfg*	*Con*

Plusarg	Tipo y rango	Valor por defecto	Componentes que lo usan	Objeto de configuración	Efecto
_TRAFFIC_PCT			*scoreboard*		*+UART_RST_EN=1, % de sesiones de tráfico que incluyen un reset a media sesión*
+UART_LCR_WLS_WEIGHTS	*Lista de 4 enteros ≥ 0 separados por comas, sin espacios (5, 6, 7 y 8 bits); al menos uno > 0*	*1,1,1,1*	*Secuencia virtual, driver serie, scoreboard*	*uart_line_cfg*	*Pesos de la longitud de carácter*
+UART_LCR_STB_PCT	*Entero 0 a 100*	*50*	*Secuencia virtual, driver serie, scoreboard*	*uart_line_cfg*	*% de configuraciones con STB = 1 (1.5 bits de parada con 5 bits de datos; 2 bits con las demás longitudes)*
+UART_LCR_PARITY_WEIGHTS	*Lista de 5 enteros ≥ 0 separados por comas, sin espacios (NONE, ODD, EVEN, STICK1, STICK0); al menos uno > 0*	*2,1,1,1,1*	*Secuencia virtual, driver serie, scoreboard*	*uart_line_cfg*	*Pesos del tipo de paridad*
+UART_LCR_BC_PCT	*Entero 0 a 100*	*5*	*Secuencia virtual, driver serie, scoreboard*	*uart_line_cfg*	*% de sesiones de tráfico que activan el break en TX (LCR[6])*
+UART_LCR_BC_LEN_MAX	*Entero 1 a 4 (tiempos de carácter)*	*2*	*Secuencia virtual, driver serie, scoreboard*	*uart_line_cfg*	*Duración máxima del break en TX*
+UART_LCR_DL_MIN	*Entero 1 a 65535*	*1*	*Secuencia virtual, driver serie, scoreboard*	*uart_line_cfg*	*Valor mínimo del divisor en el uso normal*
+UART_LCR_DL_MAX_FAST	*Entero 1 a 65535, mayor o igual que +UART_LCR_DL_MIN*	*16*	*Secuencia virtual, driver serie, scoreboard*	*uart_line_cfg*	*Máximo del rango rápido del divisor, acotado por el tiempo de simulación*
+UART_LCR_DL_MAX	*Entero 1 a 65535, mayor que +UART_LCR_DL_MAX_FAST*	*65535*	*Secuencia virtual, driver serie, scoreboard*	*uart_line_cfg*	*Máximo de la cola del divisor*
+UART_LCR_DL_TAIL_PCT	*Entero 0 a 100*	*2*	*Secuencia virtual, driver serie, scoreboard*	*uart_line_cfg*	*% de configuraciones con divisor entre +UART_LCR_DL_MAX_FAST + 1 y +UART_LCR_DL_MAX*
+UART_LCR_DL_ZERO_PCT	*Entero 0 a 100*	*1*	*Secuencia virtual, driver serie, scoreboard*	*uart_line_cfg*	*% de configuraciones con DL = 0, solo en ventanas cortas de reconfiguración*
+UART_LCR_DL_ORDER_WEIGHTS	*Lista de 3 enteros ≥ 0 separados por comas, sin espacios (DLM→DLL, solo DLL, DLL→DLM);*	*8,1,1*	*Secuencia virtual, driver serie, scoreboard*	*uart_line_cfg*	*Pesos del orden de escritura del divisor*

Plusarg	Tipo y rango	Valor por defecto	Componentes que lo usan	Objeto de configuración	Efecto
	al menos uno > 0				
+UART_FCR_TL_WEIGHTS	*Lista de 4 enteros ≥ 0 separados por comas, sin espacios (1, 4, 8 y 14 bytes); al menos uno > 0*	*1,1,1,1*	*Secuencia virtual, scoreboard*	*uart_fifo_cfg*	*Pesos del nivel de disparo de la FIFO RX*
+UART_FCR_IGN_BITS_PCT	*Entero 0 a 100*	*50*	*Secuencia virtual, scoreboard*	*uart_fifo_cfg*	*% de escrituras a FCR con valores aleatorios en los bits ignorados (FCR[0] y FCR[5:3])*
+UART_FCR_CLR_PCT	*Entero 0 a 100*	*3*	*Secuencia virtual, scoreboard*	*uart_fifo_cfg*	*% de escrituras a FCR que limpian cada FIFO (FCR[1] y FCR[2])*
+UART_TX_BURST_MIN	*Entero 1 a 20 (escrituras a THR)*	*1*	*Secuencia virtual, driver Wishbone*	*uart_env_cfg*	*Mínimo de escrituras a THR por ráfaga*
+UART_TX_BURST_MAX	*Entero 1 a 20, mayor o igual que +UART_TX_BURST_MIN*	*20*	*Secuencia virtual, driver Wishbone*	*uart_env_cfg*	*Máximo de escrituras a THR por ráfaga; de la 17 en adelante desbordan la FIFO TX*
+UART_TX_OVERFLOW_PCT	*Entero 0 a 100*	*5*	*Secuencia virtual, driver Wishbone*	*uart_env_cfg*	*% de ráfagas que desbordan la FIFO TX (de 17 a +UART_TX_BURST_MAX escrituras)*
+UART_SER_GAP_MAX	*Entero 0 a 255 (tiempos de bit)*	*20*	*Driver serie*	*uart_ser_agent_cfg*	*Máximo del hueco entre caracteres RX*
+UART_SER_PARITY_ERR_PCT	*Entero 0 a 100*	*5*	*Driver serie, scoreboard*	*uart_ser_agent_cfg*	*% de caracteres RX con paridad corrupta (solo con paridad habilitada)*
+UART_SER_FRAMING_ERR_PCT	*Entero 0 a 100*	*5*	*Driver serie, scoreboard*	*uart_ser_agent_cfg*	*% de caracteres RX con el primer bit de parada en 0*
+UART_SER_STOP2_ERR_PCT	*Entero 0 a 100*	*5*	*Driver serie, scoreboard*	*uart_ser_agent_cfg*	*% de caracteres RX con un 0 en el segundo bit de parada (solo con 1.5 o 2 bits de parada)*
+UART_SER_BREAK_PCT	*Entero 0 a 100*	*3*	*Driver serie, scoreboard*	*uart_ser_agent_cfg*	*% de huecos entre caracteres RX que se convierten en un break*
+UART_SER_BREAK_LEN_MAX	*Entero 1 a 8 (tiempos de carácter)*	*3*	*Driver serie, scoreboard*	*uart_ser_agent_cfg*	*Duración máxima del break en RX*
+UART_SER_GLITCH_PCT	*Entero 0 a 100*	*3*	*Driver serie, scoreboard*	*uart_ser_agent_cfg*	*% de huecos entre caracteres RX con un glitch de 1 a 7 ticks (de 16 por bit)*
+UART_SER_BA	*Entero 0 a 100*	*10*	*Driver serie,*	*uart_ser_agent_cf*	*% de caracteres*

Plusarg	Tipo y rango	Valor por defecto	Componentes que lo usan	Objeto de configuración	Efecto
UD_DEV_PCT			*scoreboard*	*g*	*RX con desviación de baudios*
+UART_SER_BAUD_DEV_MAX	*Entero 0 a 10 (% del periodo de bit)*	*10*	*Driver serie, scoreboard*	*uart_ser_agent_cfg*	*Desviación máxima de baudios, en el rango de −máximo a +máximo*
+UART_MODEM_TOGGLE_PCT	*Entero 0 a 100*	*20*	*Driver de módem, scoreboard*	*modem_agent_cfg*	*% de probabilidad de que cada entrada (CTS, DSR, RI y DCD) cambie por oportunidad de estímulo*
+UART_MCR_CTRL_ONE_PCT	*Entero 0 a 100*	*50*	*Secuencia virtual, driver de módem, scoreboard*	*modem_agent_cfg*	*% de probabilidad de que cada bit de MCR[3:0] valga 1*
+UART_MCR_LOOPBACK_PCT	*Entero 0 a 100*	*15*	*Secuencia virtual, driver de módem, scoreboard*	*modem_agent_cfg*	*% de configuraciones con loopback (MCR[4])*
+UART_IER_BIT_EN_PCT	*Entero 0 a 100*	*50*	*Secuencia virtual, scoreboard*	*uart_env_cfg*	*% de probabilidad de que cada bit de IER[3:0] esté habilitado*
+UART_CFG_IRQ_OVERLAP_PCT	*Entero 0 a 100*	*10*	*Secuencia virtual, scoreboard*	*uart_env_cfg*	*% de sesiones que programan coincidencia de interrupciones*
+UART_CFG_RECONFIG_PCT	*Entero 0 a 100*	*10*	*Secuencia virtual, scoreboard*	*uart_env_cfg*	*% de sesiones que solicitan una reconfiguración durante el tráfico*
+UART_SW_SERVICE_MODE_WEIGHTS	*Lista de 2 enteros ≥ 0 separados por comas, sin espacios (sondeo, servicio por interrupción); al menos uno > 0*	*1,1*	*Secuencia virtual, scoreboard*	*uart_env_cfg*	*Pesos del modo de servicio del software*
+UART_SW_SERVICE_DELAY_MIN	*Entero 0 a 255 (tiempos de carácter)*	*0*	*Secuencia virtual, scoreboard*	*uart_env_cfg*	*Mínimo del retraso de servicio*
+UART_SW_SERVICE_DELAY_FAST_MAX	*Entero 0 a 255, mayor o igual que +UART_SW_SERVICE_DELAY_MIN*	*20*	*Secuencia virtual, scoreboard*	*uart_env_cfg*	*Máximo del rango rápido del retraso de servicio*
+UART_SW_SERVICE_DELAY_MAX	*Entero 0 a 255, mayor que +UART_SW_SERVICE_DELAY_FAST_MAX*	*255*	*Secuencia virtual, scoreboard*	*uart_env_cfg*	*Máximo de la cola del retraso de servicio*
+UART_SW_SERVICE_DELAY_TAIL_PCT	*Entero 0 a 100*	*2*	*Secuencia virtual, scoreboard*	*uart_env_cfg*	*% de servicios con retraso entre +UART_SW_SERVICE_DELAY_FAST_MAX + 1 y +UART_SW_SERVICE_DELAY_MAX*

8. Casos de esquina planificados

ID	Escenario	Archivo .args	Plusargs	Bins objetivo	Seed
CE-01	*Overrun de la FIFO RX con nivel de disparo 14, 5 bits por carácter, servicio por sondeo muy lento*	*rx_overrun_tl14_5bit.args*	*+UART_FCR_TL_WEIGHTS=0,0,0,1 +UART_LCR_WLS_WEIGHTS=1,0,0,0 +UART_SW_SERVICE_MODE_WEIGHTS=1,0 +UART_SW_SERVICE_DELAY_MIN=100 +UART_SW_SERVICE_DELAY_FAST_MAX=200 +UART_SW_SERVICE_DELAY_MAX=255 +UART_SW_SERVICE_DELAY_TAIL_PCT=50 +UART_SER_GAP_MAX=0 +UART_SER_BREAK_PCT=0 +UART_SER_GLITCH_PCT=0*	*cg_err.cp_oe × cp_tl[14]; cg_rx.cp_rx_fifo_level[16] × cg_line_cfg.cp_wls[5]; cg_err.cp_lsr_b7 × cp_error_kind*	*20*
CE-02	*Overrun de la FIFO RX con cada nivel de disparo y 8 bits por carácter*	*rx_overrun_all_tl_8bit.args*	*+UART_FCR_TL_WEIGHTS=1,1,1,1 +UART_LCR_WLS_WEIGHTS=0,0,0,1 +UART_SW_SERVICE_MODE_WEIGHTS=1,0 +UART_SW_SERVICE_DELAY_MIN=150 +UART_SW_SERVICE_DELAY_FAST_MAX=200 +UART_SW_SERVICE_DELAY_MAX=255 +UART_SER_GAP_MAX=0*	*cg_err.cp_oe × cp_tl (4 bins); cg_rx.cp_rx_fifo_level[16] × cp_wls[8]*	*20*
CE-03	*Desbordamiento de la FIFO TX en todas las ráfagas (17 a 20 escrituras a THR)*	*tx_overflow_always.args*	*+UART_TX_BURST_MIN=17 +UART_TX_BURST_MAX=20 +UART_TX_OVERFLOW_PCT=100*	*cg_tx.cp_tx_fifo_level[16] × cp_overflow; cg_tx.cp_tx_fifo_level[0:16]; cg_tx.cp_tx_fifo_level[0] × cp_tx_busy*	*10*
CE-04	*Ráfaga mínima de una sola escritura a THR sin desbordamiento (THRE y TEMT en vacío)*	*tx_single_write.args*	*+UART_TX_BURST_MIN=1 +UART_TX_BURST_MAX=1 +UART_TX_OVERFLOW_PCT=0 +UART_IER_BIT_EN_PCT=100*	*cg_tx.cp_tx_fifo_level[0:2] × cg_line_cfg.cp_wls; cg_int.cp_thre_event × cp_ier_enable*	*10*
CE-05	*Divisor máximo de la cola (65534 a 65535) con ráfagas cortas por tiempo de simulación*	*dl_max_tail.args*	*+UART_LCR_DL_MIN=65534 +UART_LCR_DL_MAX_FAST=65534 +UART_LCR_DL_MAX=65535 +UART_LCR_DL_TAIL_PCT=50 +UART_LCR_DL_ZERO_PCT=0 +UART_TX_BURST_MAX=2 +UART_TX_OVERFLOW_PCT=0 +UART_SER_GAP_MAX=0*		
CE-06	*Divisor mínimo (DL = 1 y 2) con 8 bits y 2 bits de parada: máxima tasa de baudios*	*dl_min_fast.args*	*+UART_LCR_DL_MIN=1 +UART_LCR_DL_MAX_FAST=1 +UART_LCR_DL_MAX=2 +UART_LCR_DL_TAIL_PCT=50 +UART_LCR_DL_ZERO_PCT=0 +UART_LCR_WLS_WEIGHTS=0,0,0,1 +UART_LCR_STB_PCT=100*	*cg_line_cfg.cp_dl[1, 2]; cover de la aserción de baud_o (V-IO-02); cg_line_cfg.cp_wls × cp_stb*	*20*
CE-07	*Divisor en cero durante ventanas cortas de reconfiguración*	*dl_zero_reconfig.args*	*+UART_LCR_DL_ZERO_PCT=30 +UART_CFG_RECONFIG_PCT=100 +UART_TX_BURST_MAX=4*	*cg_line_cfg.cp_dl[ZERO]; cg_reconfig.cp_dl_order*	*20*
CE-08	*Reconfiguración del divisor en los tres órdenes de escritura (DLM→DLL, solo DLL, DLL→DLM)*	*dl_write_orders.args*	*+UART_LCR_DL_ORDER_WEIGHTS=1,1,1 +UART_CFG_RECONFIG_PCT=100 +UART_LCR_DL_MAX_FAST=16*	*cg_reconfig.cp_dl_order (3 bins); cg_regs.cp_init_seq*	*20*
CE-09	*Break en RX de 1 a 8 tiempos de carácter en todos los huecos*	*rx_break_max.args*	*+UART_SER_BREAK_PCT=100 +UART_SER_BREAK_LEN_MAX=8 +UART_SER_GAP_MAX=2 +UART_IER_BIT_EN_PCT=100*	*cg_err.cp_bi × cp_break_len; cg_int.cp_rx_error × cp_iir_id[RLS]; cg_err.cp_lsr_b7*	*20*

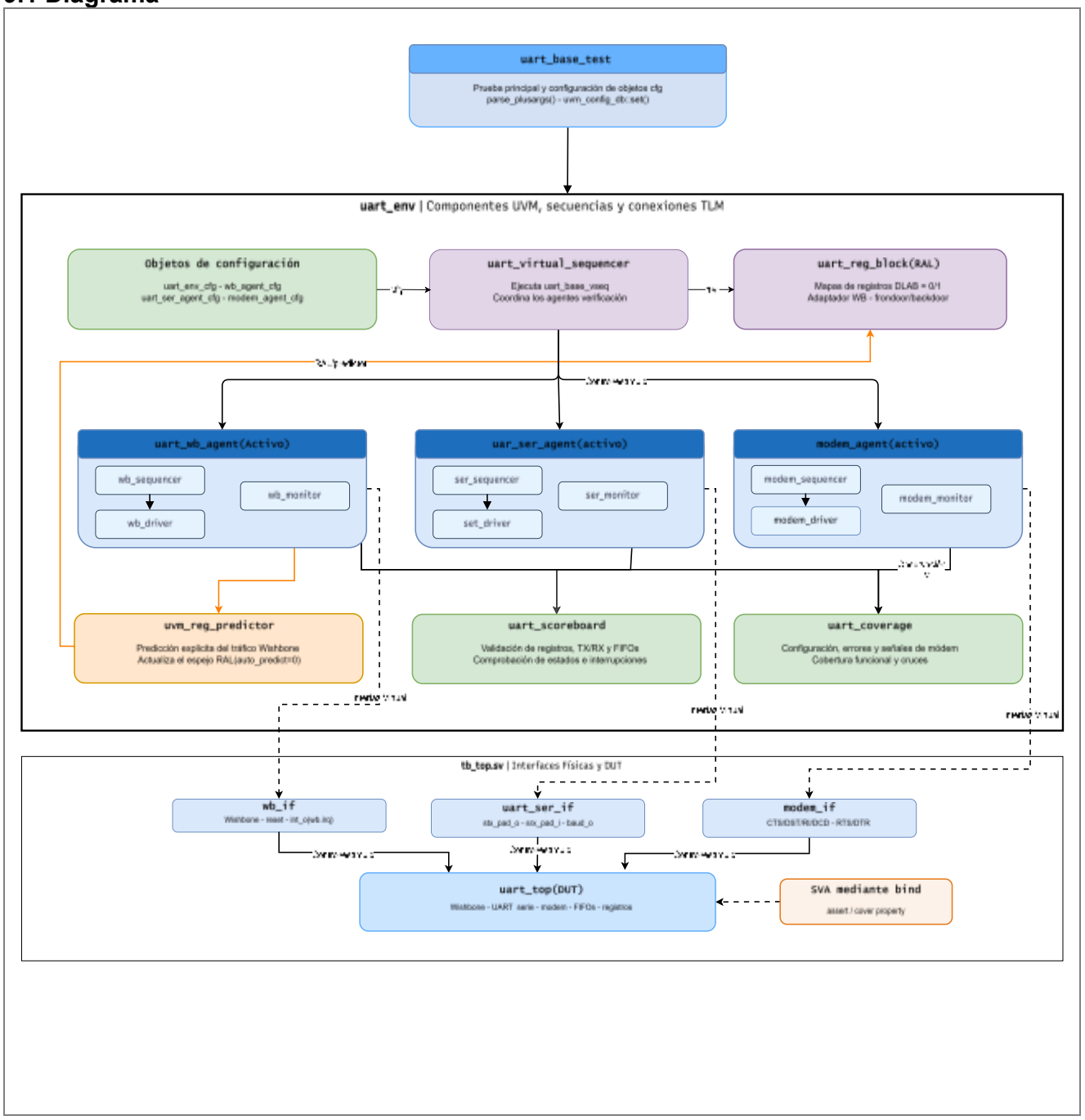
ID	Escenario	Archivo .args	Plusargs	Bins objetivo	Seed
				× cp_error_kind	
CE-10	Break en TX de 1 a 4 tiempos de carácter en todas las sesiones	tx_break_max.args	+UART_LCR_BC_PCT=100 +UART_LCR_BC_LEN_MAX=4	cg_line_cfg.cp_bc; cover de A-SER-04	10
CE-11	Error de framing en todos los caracteres RX con cada longitud de carácter	rx_framing_all_wls.args	+UART_SER_FRAMING_ERR_PCT=100 +UART_SER_PARITY_ERR_PCT=0 +UART_SER_STOP2_ERR_PCT=0 +UART_SER_BREAK_PCT=0 +UART_LCR_WLS_WEIGHTS=1,1,1,1	cg_err.cp_fe × cg_line_cfg.cp_wls (4 bins)	20
CE-12	Segundo bit de parada en 0 con 1.5 y 2 bits de parada	rx_stop2_err.args	+UART_LCR_STB_PCT=100 +UART_SER_STOP2_ERR_PCT=100 +UART_SER_FRAMING_ERR_PCT=0 +UART_SER_PARITY_ERR_PCT=0 +UART_SER_BREAK_PCT=0	cg_err.cp_stop2_err; cg_line_cfg.cp_stb × cp_wls	10
CE-13	Error de paridad en todos los caracteres RX con los cuatro tipos de paridad habilitada	rx_parity_err_all.args	+UART_LCR_PARITY_WEIGHTS=0,1,1,1,1 +UART_SER_PARITY_ERR_PCT=100 +UART_SER_FRAMING_ERR_PCT=0 +UART_SER_STOP2_ERR_PCT=0 +UART_SER_BREAK_PCT=0	cg_err.cp_pe × cg_line_cfg.cp_parity (ODD, EVEN, STICK1, STICK0)	20
CE-14	Paridad stick 0 y stick 1 sin errores inyectados	parity_stick_clean.args	+UART_LCR_PARITY_WEIGHTS=0,0,0,1,1 +UART_SER_PARITY_ERR_PCT=0	cg_line_cfg.cp_parity[STICK0, STICK1] × cp_dir	10
CE-15	Desviación de baudios máxima (±10 %) en todos los caracteres RX	rx_baud_dev_max.args	+UART_SER_BAUD_DEV_PCT=100 +UART_SER_BAUD_DEV_MAX=10 +UART_SER_GAP_MAX=0	cg_ser.cp_baud_dev (extremos −10 % y +10 %)	20
CE-16	Glitch en todos los huecos entre caracteres RX (1 a 7 ticks)	rx_glitch_always.args	+UART_SER_GLITCH_PCT=100 +UART_SER_BREAK_PCT=0 +UART_SER_GAP_MAX=20	cg_ser.cp_glitch	10
CE-17	Reset a media sesión en todas las sesiones, cubriendo las cinco fases	reset_mid_traffic_all.args	+UART_RST_EN=1 +UART_RST_MID_TRAFFIC_PCT=100	cg_rst.cp_reset_phase (reposo, TX activo, RX activo, FIFO llena, DLAB = 1); A-RST-01	30
CE-18	Accesos Wishbone con wb_sel_ilegal (0 bits o 2 o más bits)	wb_sel_illegal.args	+UART_WB_SEL_ILLEGAL_PCT=50 +UART_WB_SEL_WEIGHTS=1,1,1,1	cg_bus.cp_sel_illegal; cg_bus.cp_sel × cp_dir	10
CE-19	Retardos Wishbone en el máximo: hasta 255 ciclos entre transacciones y entre cyc y stb	wb_delay_long.args	+UART_WB_DELAY_MAX=255 +UART_WB_STB_DELAY_MAX=255	cg_bus.cp_ack_latency; A-WB-01 a A-WB-04 (assert y cover)	5
CE-20	Accesos solo a registros Debug y direcciones no mapeadas, con DLAB = 1 la mitad del tiempo	reg_debug_unmapped.args	+UART_REG_ADDR_WEIGHTS=0,1,1 +UART_REG_DLAB_PCT=50	cg_regs.cp_addr[8, 12, no mapeadas] × cp_dir; cg_regs.cp_addr[8, 12] × cp_dir × cp_dbg_core_state	10
CE-21	Accesos a registros normales	reg_dlab_always.args	+UART_REG_ADDR_WEIGHTS=1,0,0 +UART_REG_DLAB_PCT=100	cg_regs.cp_addr × cp_dir × cp_dlab; V-REG-	10

ID	Escenario	Archivo .args	Plusargs	Bins objetivo	Seed
	siempre con DLAB = 1 (divisor accesible; RBR, THR e IER no)			*03 y V-LCR-07*	
CE-22	*Coincidencia de todas las interrupciones con IER[3:0] = 1111*	*irq_overlap_all.args*	*+UART_CFG_IRQ_OVERLAP_PCT=100 +UART_IER_BIT_EN_PCT=100 +UART_MODEM_TOGGLE_PCT=60 +UART_SER_PARITY_ERR_PCT=20 +UART_SER_FRAMING_ERR_PCT=20*	*cg_int.cp_priority_pair × cp_iir_id; cg_int.cp_iir_id × cp_service_action × cp_pending_after*	*30*
CE-23	*Timeout de recepción: servicio por interrupción, huecos largos entre caracteres RX y niveles de disparo 4, 8 y 14*	*rx_timeout_long_gap.args*	*+UART_SW_SERVICE_MODE_WEIGHTS=0,1 +UART_FCR_TL_WEIGHTS=0,1,1,1 +UART_SER_GAP_MAX=255 +UART_IER_BIT_EN_PCT=100*	*cg_int.cp_iir_id[TIMEOUT] × cp_fifo_trigger; cg_int.cp_iir_id × cp_service_action × cp_pending_after (TIMEOUT, READ_RBR)*	*20*
CE-24	*RDA se limpia al bajar de nivel de disparo: servicio inmediato por sondeo*	*rx_rda_fast_service.args*	*+UART_SW_SERVICE_MODE_WEIGHTS=1,0 +UART_SW_SERVICE_DELAY_MIN=0 +UART_SW_SERVICE_DELAY_FAST_MAX=0 +UART_SW_SERVICE_DELAY_MAX=1 +UART_SW_SERVICE_DELAY_TAIL_PCT=0 +UART_FCR_TL_WEIGHTS=1,1,1,1*	*cg_int.cp_fifo_trigger × cp_fifo_below_trigger × cp_pending_after; cg_int.cp_iir_id[RDA] × cp_fifo_trigger*	*20*
CE-25	*Limpieza constante de FIFO RX y TX con bits ignorados de FCR siempre aleatorios*	*fcr_clear_always.args*	*+UART_FCR_CLR_PCT=100 +UART_FCR_IGN_BITS_PCT=100*	*cg_fifo_cfg.cp_rx_clr × cp_rx_busy; cg_fifo_cfg.cp_tx_clr × cp_tx_busy; cg_fifo_cfg.cp_fcr_b0; cg_fifo_cfg.cp_fcr_b5_3*	*20*
CE-26	*Cambios constantes en CTS, DSR, RI y DCD con interrupción de módem habilitada (sin loopback)*	*modem_toggle_max.args*	*+UART_MODEM_TOGGLE_PCT=100 +UART_MCR_LOOPBACK_PCT=0 +UART_IER_BIT_EN_PCT=100*	*cg_modem.cp_cts_edge × cp_dcts; cp_dsr_edge × cp_ddsr; cp_ri_edge × cp_teri; cp_dcd_edge × cp_ddcd; cg_int.cp_modem_source × cp_iir_id[MODEM]*	*20*
CE-27	*Loopback siempre activo con salidas MCR aleatorias y tráfico serie; las entradas externas de módem se ignoran*	*mcr_loopback_always.args*	*+UART_MCR_LOOPBACK_PCT=100 +UART_MCR_CTRL_ONE_PCT=50 +UART_MODEM_TOGGLE_PCT=50*	*cg_modem.cp_loopback × cp_out1; cp_loopback × cp_out2; cp_loopback × cp_serial_activity; cg_modem.cp_loopback × cp_signal × cp_level*	*20*
CE-28	*Configuración mínima del tráfico: 5 bits, sin paridad, 1 bit de parada, DL = 1*	*min_config.args*	*+UART_LCR_WLS_WEIGHTS=1,0,0,0 +UART_LCR_STB_PCT=0 +UART_LCR_PARITY_WEIGHTS=1,0,0,0,0 +UART_LCR_DL_MIN=1 +UART_LCR_DL_MAX_FAST=1 +UART_LCR_DL_MAX=2 +UART_LCR_DL_TAIL_PCT=0 +UART_LCR_DL_ZERO_PCT=0*	*cg_line_cfg.cp_wls[5] × cp_stb; cg_line_cfg.cp_parity[NONE]; cg_line_cfg.cp_dl[1]*	*10*

ID	Escenario	Archivo .args	Plusargs	Bins objetivo	Seed
CE-29	*Variante de bus de 8 bits (requiere compilar con +define+DATA_BUS_WIDTH_8)*	*bus8_variant.args*	*+UART_WB_BUS_WIDTH=8 +UART_REG_ADDR_WEIGHTS=1,0,0*	*cg_regs.cp_addr[0:6] en la regresión de la variante; V-GEN-03*	*10*

9. Arquitectura del ambiente

9.1 Diagrama



9.2 Componentes

Componente	Clase UVM base	Responsabilidad	Puertos TLM
uart_wb_agent	*uvm_agent*	*Maneja y observa la interfaz Wishbone*	*ap (uvm_analysis_port)*
uart_base_test	*uvm_test*	*Crea uart_env interpreta los plusargs y configura los objetos*	*No requiere puertos propios*
uart_env	*uvm_env*	*Integra los tres agentes, secuenciador, RAL y los demás componentes. Realiza sus conexiones en connect_phase.*	*Conecta los puertos de análisis de los monitores con predictor, scoreboard y cobertura.*
uart_virtual_sequencer	*uvm_sequencer*	*Mantiene referencias a los tres componentes principales para*	*Referencias wb_seqr , ser_seqr y modem_seqr; no*

Componente	Clase UVM base	Responsabilidad	Puertos TLM
		coordinar las operaciones de uart_base_vseq	*necesita puertos de análisis propios.*
uart_wb_agent	*uvm_agent*	*Genera y observa accesos Wishbone.*	*seq_item_port/export entre driver y secuenciador; ap del monitor hacia predictor, scoreboard y cobertura.*
uart_ser_agent	*uvm_agent*	*Genera los estímulos RX y observar las tramas TX/RX*	*seq_item_port/export; puertos de análisis tx_ap y rx_ap hacia scoreboard y cobertura*
modem_agent	*uvm_agent*	*Controla CTS, DSR, RI y DCD; observa RTS, DTR y int_o.*	*seq_item_port/export; puertos de análisis modem_ap e irq_ap hacia scoreboard y cobertura.*
uart_reg_block	*uvm_reg_block*	*Modela los registros UART, sus campos, accesos, reset y mapas asociados a DLAB = 0/1.*	*No tiene puertos TLM propios; utiliza el adaptador Wishbone y el secuenciador correspondiente.*
uart_wb_reg_adapter	*uvm_reg_adapter*	*Convierte operaciones RAL a transacciones Wishbone mediante reg2bus() y respuestas Wishbone a operaciones RAL mediante bus2reg().*	*No tiene puertos TLM propios; se asocia al mapa RAL y al secuenciador Wishbone.*
uart_scoreboard	*uvm_scoreboard*	*Mantiene un modelo de referencia y compara TX, RX, FIFOs, LSR, IIR, MSR, int_o y los efectos de las operaciones de registro.*	*Entradas de análisis mediante uvm_analysis_imp o uvm_tlm_analysis_fifo para transacciones Wishbone, serie, módem e interrupciones.*
uart_coverage	*uvm_component*	*Recopila cobertura funcional de configuraciones, registros, tráfico, errores, FIFO, módem e interrupciones según la sección 13.*	*Entradas de análisis conectadas a los monitores y, cuando corresponda, al estado del modelo de referencia.*

9.3 Flujo de configuración

Objeto de configuración	Ruta en uvm_config_db	Lo publica	Lo obtienen
uart_env_cfg	*uvm_test_top.env*	*uart_base_test*	*uart_env; su configuración se comparte con el secuenciador virtual, scoreboard y cobertura.*
wb_agent_cfg	*uvm_test_top.env.wb_agt**	*uart_base_test*	*uart_wb_agent, driver, monitor y secuenciador Wishbone.*
uart_ser_agent_cfg	*uvm_test_top.env.ser_agt**	*uart_base_test*	*uart_ser_agent, driver, monitores TX/RX y*

Objeto de configuración	Ruta en uvm_config_db	Lo publica	Lo obtienen
			secuenciador serie.
modem_agent_cfg	*uvm_test_top.env.mdm_agt**	*uart_base_test*	*modem_agent, driver, monitor de módem, monitor de interrupciones y secuenciador.*
wb_vif (virtual wb_if)	*uvm_test_top**	*tb_top.sv*	*uart_base_test; se distribuye al agente Wishbone y al monitor de interrupciones.*
ser_vif (virtual uart_ser_if)	*uvm_test_top**	*tb_top.sv*	*uart_base_test; se distribuye al driver y a los monitores serie.*
mdm_vif (virtual modem_if)	*uvm_test_top**	*tb_top.sv*	*uart_base_test; se distribuye al driver y al monitor de módem.*

10. Modelo de referencia y scoreboard

Qué se predice	Fuente de la predicción	Momento de la comparación	Tolerancia
Datos TX	*Escrituras a THR (monitor Wishbone)*	*Al reconstruir el carácter en stx_pad_o*	*Ninguna*
Datos RX	*Tramas observadas en srx_pad_i configuración LCR/DL y modelo de FIFO RX con los errores asociados a cada carácter*	*Al completar una lectura válida de RBR mediante Wishbone, comparando el dato devuelto con el primer carácter esperado de la FIFO*	*Ninguna, se consideran las reglas de overrun y limpieza de FIFO*
LSR	*Estado esperado de FIFOs TX/RX errores de recepción, estado del transmisor escrituras THR y lecturas de LSR*	*En cada lectura válida de LSR y después de los eventos que modifican sus indicadores, respetando la actualización temporal del estado*	*Sin tolerancia en los valores de los bits. Se considera el retardo documentado de THRE y la actualización sincronizada de los indicadores*
IIR	*Modelo independiente de interrupciones basado en IER, condiciones RLS/RDA/timeout/THRE/Modem Status, prioridades y mecanismos de limpieza*	*En cada lectura válida de IIR, utilizando las fuentes pendientes esperadas y el orden de prioridad*	*Ninguna diferencia en el identificador, bit de pendiente o bits fijos. Las comparaciones se realizan considerando la latencia de actualización de los registros*
MSR	*Transacciones observadas de CTS, DRS, RI y DCD; configuración MCR; modo loopback; historial de señales e indicadores de cambio*	*En cada válida la MSR, comparando el estado actual y los indicadores acumulados antes de aplicas el efecto de limpieza de esa lectura*	*No hay ninguna diferencia en los bits esperados. Se contempla la actualización sincronizada y se documentan las discrepancias del AMB-03 y AMB-04*
int_o	*Modelo de interrupciones pendientes y habilitadas mediante IER, incluyendo RLS, RDA, timeout, THRE y Modem Status y sus condiciones de atención*	*Al producirse cambios de fuentes pendientes, habilitaciones o acciones de servicio, observando int_o mediante el monitor de interrupciones*	*Sin tolerancia en el nivel lógico esperado. Se permitirá únicamente una ventana temporal acotada por la latencia del RTL, definida antes de implementas comparaciones ciclo a ciclo*

11. Aserciones

ID	Interfaz	Propiedad (en palabras)	Tipo (assert, assume, cover)	Ítem del plan
A-WB-01	*Wishbone*	*adr, dat_i, we y sel estables mientras cyc y stb están activos y no hay ack*	*assert y cover*	*V-WB-03*
A-WB-02	*Wishbone*	*ack solo se activa si cyc y stb están activos*	*assert y cover*	*V-WB-03*
A-WB-03	*Wishbone*	*Toda transferencia con cyc y stb activos recibe ack exactamente 2 ciclos después de que se observa la solicitud*	*assert y cover*	*V-WB-03*
A-WB-04	*Wishbone*	*En una lectura, dat_o no tiene valores X cuando llega ack*	*assert y cover*	*V-WB-03*
A-SER-01	*Serie*	*Sin break ni loopback, cuando el transmisor está vacío (LSR[6] = 1), stx está en 1 (reposo), incluido después del reset*	*assert y cover*	*V-IO-03*
A-SER-02	*Serie*	*Los bits de inicio, datos y paridad duran 16 × DL ciclos cada uno; los de parada duran 16, 24 o 32 × DL según LCR*	*assert y cover*	*V-DL-03*
A-SER-03	*Serie*	*Con LCR[6] = 1 y sin loopback, stx está en 0 (break)*	*assert y cover*	*V-LCR-06*
A-SER-04	*Serie*	*srx nunca tiene valores X o Z*	*assume*	*V-IO-03*
A-SER-05	*Serie*	*Con DL = 0 (a partir del segundo ciclo en que vale 0), stx no cambia: no arranca ninguna trama*	*assert y cover*	*V-DL-02*
A-SER-06	*Serie*	*Con UART_HAS_BAUDRATE_OUTPUT, baud_o hace un pulso de 1 ciclo cada DL ciclos (con DL = 1 queda siempre en 1)*	*assert y cover*	*V-IO-02*
A-MOD-01	*Módem*	*Con loopback activo (MCR[4] = 1), stx se mantiene en 1*	*assert y cover*	*V-MCR-05*
A-MOD-02	*Módem*	*rts_pad_o y dtr_pad_o reflejan MCR[1] y MCR[0] según la polaridad elegida en AMB-03*	*assert y cover*	*V-MCR-01, V-MCR-02*
A-MOD-03	*Módem*	*Fuera de loopback, un cambio en cts_pad_i activa MSR[0] (DCTS) en el ciclo siguiente, salvo que coincida con la limpieza por lectura de MSR*	*assert y cover*	*V-MSR-01*
A-MOD-04	*Módem*	*cts, dsr, ri y dcd nunca tienen valores X o Z*	*assume*	*V-IO-04*
A-MOD-05	*Módem*	*Con loopback activo, en el ciclo siguiente*	*assert y cover*	*V-MCR-06*

ID	Interfaz	Propiedad (en palabras)	Tipo (assert, assume, cover)	Ítem del plan
		MSR[7:4] refleja {OUT2, OUT1, DTR, RTS} (MCR[3], MCR[2], MCR[0], MCR[1]) y no las entradas externas		
A-INT-01	*Interrupción*	*Con IER = 0, int_o se mantiene en 0 a partir del segundo ciclo después de escribir IER*	*assert y cover*	*V-IO-05*
A-INT-02	*Interrupción*	*Si int_o = 1, entonces IIR[0] = 0 en el mismo ciclo; al atender una fuente, int_o puede bajar un ciclo antes que IIR*	*assert y cover*	*V-INT-01, V-IO-01*
A-INT-03	*Interrupción*	*int_o se activa al menos una vez por cada causa: RLS, RDA, timeout, THRE y MS*	*cover*	*V-INT-03 a V-INT-07*
A-RST-01	*Reset*	*Durante el reset, int_o = 0 y stx = 1*	*assert y cover*	*V-RST-01*

12. RAL

12.1 Registros y campos

Registro	Dirección	DLAB	Campo	Bits	Acceso	Reset	Ruta de puerta trasera
Receiver Buffer (RBR)	*0*	*0 (lectura)*	*rbr_data*	*7:0*	*RO*	*--*	*--*
Transmitter Holding (THR)	*0*	*0 (escritura)*	*thr_data*	*7:0*	*WO*	*--*	*--*
Interrupt Enable (IER)	*1*	*0*	*erbfi*	*0*	*RW*	*0*	*uart_top.regs.ier[0]*
Interrupt Enable (IER)	*1*	*0*	*etbei*	*1*	*RW*	*0*	*uart_top.regs.ier[1]*
Interrupt Enable (IER)	*1*	*0*	*elsi*	*2*	*RW*	*0*	*uart_top.regs.ier[2]*
Interrupt Enable (IER)	*1*	*0*	*edssi*	*3*	*RW*	*0*	*uart_top.regs.ier[3]*
Interrupt Enable (IER)	*1*	*0*	*reserved*	*7:4*	*RW*	*0*	*--*
Interrupt Identification (IIR)	*2*	*0*	*int_pending*	*0*	*RO*	*1*	*uart_top.regs.iir[0]*
Interrupt Identification (IIR)	*2*	*0*	*int_id*	*3:1*	*RO*	*3'b000*	*uart_top.regs.iir[3:1]*
Interrupt Identification (IIR)	*2*	*0*	*reserved*	*5:4*	*RO*	*2'b00*	*--*
Interrupt Identification (IIR)	*2*	*0*	*fifo_en*	*7:6*	*RO*	*2'b11*	*--*
FIFO Control (FCR)	*2*	*0*	*reserved*	*0*	*W*	*0*	*--*
FIFO Control (FCR)	*2*	*0*	*rx_fifo_rst*	*1*	*W*	*0*	*uart_top.regs.rx_reset*
FIFO Control (FCR)	*2*	*0*	*tx_fifo_rst*	*2*	*W*	*0*	*uart_top.regs.tx_reset*
FIFO Control (FCR)	*2*	*0*	*reserved*	*5:3*	*W*	*3'b000*	*--*
FIFO Control (FCR)	*2*	*0*	*rx_trig*	*7:6*	*W*	*2'b11*	*uart_top.regs.fcr[1:0]*
Line Control (LCR)	*3*	*0 y 1*	*WLS*	*1:0*	*RW*	*2'b11*	*uart_top.regs.lcr[1:0]*
Line Control (LCR)	*3*	*0 y 1*	*SSB*	*2*	*RW*	*0*	*uart_top.regs.lcr[2]*
Line Control (LCR)	*3*	*0 y 1*	*PAE*	*3*	*RW*	*0*	*uart_top.regs.lcr[3]*
Line Control (LCR)	*3*	*0 y 1*	*EPS*	*4*	*RW*	*0*	*uart_top.regs.lcr[4]*
Line Control (LCR)	*3*	*0 y 1*	*SP*	*5*	*RW*	*0*	*uart_top.regs.lcr[5]*
Line Control (LCR)	*3*	*0 y 1*	*BC*	*6*	*RW*	*0*	*uart_top.regs.lcr[6]*
Line Control (LCR)	*3*	*0 y 1*	*DLAB*	*7*	*RW*	*0*	*uart_top.regs.lcr[7]*
Modem Control (MCR)	*4*	*0 y 1*	*DTR*	*0*	*WO*	*0*	*uart_top.regs.mcr[0]*
Modem Control (MCR)	*4*	*0 y 1*	*RTS*	*1*	*WO*	*0*	*uart_top.regs.mcr[1]*
Modem Control (MCR)	*4*	*0 y 1*	*OUT1*	*2*	*WO*	*0*	*uart_top.regs.mcr[2]*
Modem Control (MCR)	*4*	*0 y 1*	*OUT2*	*3*	*WO*	*0*	*uart_top.regs.mcr[3]*
Modem Control (MCR)	*4*	*0 y 1*	*LOOP*	*4*	*WO*	*0*	*uart_top.regs.mcr[4]*
Modem Control (MCR)	*4*	*0 y 1*	*reserved*	*7:5*	*WO*	*0*	*--*
Line Status (LSR)	*5*	*0 y 1*	*DR*	*0*	*RO*	*0*	*uart_top.regs.lsr[0]*
Line Status (LSR)	*5*	*0 y 1*	*OE*	*1*	*RC*	*0*	*uart_top.regs.lsr[1]*
Line Status (LSR)	*5*	*0 y 1*	*PE*	*2*	*RC*	*0*	*uart_top.regs.lsr[2]*

Registro	Dirección	DLAB	Campo	Bits	Acceso	Reset	Ruta de puerta trasera
Line Status (LSR)	*5*	*0 y 1*	*FE*	*3*	*RC*	*0*	*uart_top.regs.lsr[3]*
Line Status (LSR)	*5*	*0 y 1*	*BI*	*4*	*RC*	*0*	*uart_top.regs.lsr[4]*
Line Status (LSR)	*5*	*0 y 1*	*THRE*	*5*	*RO*	*1*	*uart_top.regs.lsr[5]*
Line Status (LSR)	*5*	*0 y 1*	*TEMT*	*6*	*RO*	*1*	*uart_top.regs.lsr[6]*
Line Status (LSR)	*5*	*0 y 1*	*RXFE*	*7*	*RC*	*0*	*uart_top.regs.lsr[7]*
Modem Status (MSR)	*6*	*0 y 1*	*DCTS*	*0*	*RC*	*0*	*uart_top.regs.msr[0]*
Modem Status (MSR)	*6*	*0 y 1*	*DDSR*	*1*	*RC*	*0*	*uart_top.regs.msr[1]*
Modem Status (MSR)	*6*	*0 y 1*	*TERI*	*2*	*RC*	*0*	*uart_top.regs.msr[2]*
Modem Status (MSR)	*6*	*0 y 1*	*DDCD*	*3*	*RC*	*0*	*uart_top.regs.msr[3]*
Modem Status (MSR)	*6*	*0 y 1*	*CTS*	*4*	*RO*	*0*	*uart_top.regs.msr[4]*
Modem Status (MSR)	*6*	*0 y 1*	*DSR*	*5*	*RO*	*0*	*uart_top.regs.msr[5]*
Modem Status (MSR)	*6*	*0 y 1*	*RI*	*6*	*RO*	*0*	*uart_top.regs.msr[6]*
Modem Status (MSR)	*6*	*0 y 1*	*DCD*	*7*	*RO*	*0*	*uart_top.regs.msr[7]*
Divisor Latch LSB (DLL)	*0*	*1*	*dll*	*7:0*	*RW*	*8'h00*	*uart_top.regs.dl[7:0]*
Divisor Latch MSB (DLM)	*1*	*1*	*dlm*	*7:0*	*RW*	*8'h00*	*uart_top.regs.dl[15:8]*
Debug 1	*8*	*--*	*lsr_val*	*7:0*	*RO*	*--*	*uart_top.regs.lsr*
Debug 1	*8*	*--*	*ier_val*	*11:8*	*RO*	*--*	*uart_top.regs.ier*
Debug 1	*8*	*--*	*iir_val*	*15:12*	*RO*	*--*	*uart_top.regs.iir*
Debug 1	*8*	*--*	*lcr_val*	*23:16*	*RO*	*--*	*uart_top.regs.lcr*
Debug 1	*8*	*--*	*msr_val*	*31:24*	*RO*	*--*	*uart_top.regs.msr*
Debug 2	*12*	*--*	*tstate*	*2:0*	*RO*	*--*	*uart_top.regs.tstate*
Debug 2	*12*	*--*	*tf_count*	*7:3*	*RO*	*--*	*uart_top.regs.tf_count*
Debug 2	*12*	*--*	*rstate*	*11:8*	*RO*	*--*	*uart_top.regs.rstate*
Debug 2	*12*	*--*	*rf_count*	*16:12*	*RO*	*--*	*uart_top.regs.rf_count*
Debug 2	*12*	*--*	*mcr_val*	*21:17*	*RO*	*--*	*uart_top.regs.mcr*
Debug 2	*12*	*--*	*fcr_val*	*23:22*	*RO*	*--*	*uart_top.regs.fcr*
Debug 2	*12*	*--*	*reserved*	*31:24*	*RO*	*--*	*--*
Scratch (SCR)	*7*	*0 y 1*	*scratch*	*7:0*	*RW*	*8'h00*	*uart_top.regs.scratch*

12.2 Decisiones de modelado

Tema	Decisión	Justificación
Registros con dirección compartida (DLAB)	*Definir un solo bloque de registros (uart_reg_block) con dos mapas, uno por cada valor de DLAB* *map_dlab0: RBR, THR, IER, IIR, FCR, LCR, MCR, LSR, MSR, SCR, Debug1 y Debug 2* *Map_dlab1: DLL(0), DLM(1), LCR, MCR, LSR, MSR Y SCR* *Los registros comunes son la misma instancia en ambos mapas con el mismo*	*Ambos mapas usan las direcciones reales del bus. Un mismo offset corresponde a dos registros distintos según LCR.DLAB (RBR/THR/IER con DLAB=0 y DLL/DLM con DLAB=1), y un mapa no admite dos registros de la misma clase en un offset.* *Separar los mapas deja un registro por dirección en cada uno, y el adaptador,*

Tema	Decisión	Justificación
	offset. *Antes de usa map_dlab1 en LCR.DLAB se debe de poner en 1y se baja a 0 al terminar*	*igual para ambos, no necesita conocer DLAB.* *La elección del mapa correcto la hacen el predictor y la secuencia base, según el mirror de LCR.DLAB.*
Predictor y adaptador: conexión y política de predicción	*Se elige un predictor explicito para (uvm_reg_predictor) que se conecta directamente al analysis_port del monitor wishbone.* *Se asocian ambos mapas al mismo adaptador. Convierte el offset en dirección Wishbone y calcula wb_sel_i según el carril del byte. o* *El encargado de elegir el mapa es el predictor según LCR.DLAB* *Se escribe en Mirror los valores de defecto cuando se aplica un reset*	*Varios registros cambian por hardware (LSR, IIR, MSR), y el tráfico de uart_base_test puede acceder al bus sin pasar por el RAL; solo un predictor conectado al monitor ve todos los accesos efectivos, sin tomar datos erroneos del modelo.* *Es el mismo monitor que alimenta al scoreboard, y deja el mirror independiente de lo que dice el driver.* *Como el bus no lleva DLAB, el predictor lo toma del mirror de LCR para saber a qué registro pertenece cada acceso a las direcciones 0 y 1.*
Efectos laterales (lectura que limpia, FIFOs)	*Se establecen los registros como RC a los OE, PE, FE, BI en LSR; DCTS, DDSR, TERI, DDCD en MSR. Que la especificación establece que se limpian al ser leídos.* *Se establece los registros RO a DR, THRE, TEMT, int_pending, int_id, CTS, DSR, RI, DCD ya que cambian por el hardware, no al ser leidos.* *FIFOs: no se modelan en el RAL, RBR y THR quedan como registros de paso. Este trabajo se le delega al scoreboard y se compara con los valores que se obtienen de los regs de Debug.* *Los registros de rx_fifo_rst/tx_fifo_rst: realizan un callback post_predict, para el scoreboard que vaciel el modelo* *Se observa LSR y MSR por backdoor para no hacerles clear.*	*Las políticas RC y volatile son los mecanismos nativos del RAL para estos casos.* *Las FIFOs no son memorias direccionables, así que no encajan en el RAL por eso se deja al scoreboard.* *El backdoor no dispara efectos laterales, por lo que sirve para revisar sin alterar el DUT.*
Secuencias integradas que se correrán y exclusiones	*Se corren uvm_reg_hw_reset_seq, uvm_reg_bit_bash_seq, uvm_reg_access_seq y uvm_reg_shared_access_seq, justo después del reset y antes del tráfico.* *El reset se va a correr en todos los registros menos en RBR, THR, FCR y los Debug.* *El acceso a backdoor se corre solo en los que se define.* *Para escribir o leer con, bit_bash se va a corre sobre SCR, IER (bits 3:0) y DLL/DLM; se excluyen LSR, MSR, IIR, RBR, THR, FCR, MCR y Debug por ser RO/WO/RC o volátiles.*	*Cada una de las exclusión ser hace por el tipo de acceso o el efecto lateral (RO/WO, RC, volátil o que no hay ruta de acceso)*

13. Modelo de cobertura funcional

Covergroup	Dónde se muestrea	Coverpoints	Cruces	Bins ilegales o ignorados (justificación)
cg_line_cfg	*Monitor Wishbone al escribir LCR*	*wls, stb, pen, eps, sp, bc*	*wls × stb × paridad*	*Stick sin paridad habilitada: ignorado, no tiene efecto*
cg_regs	*uart_coverage: transacciones Wishbone confirmadas con ACK; lecturas Debug 1/2 y contexto del modelo; fin de inicialización y comprobaciones tras reset.*	*cp_addr: bins individuales 0–7, 8, 12 y no mapeadas (9–11, 13–31) en bus 32 bits; cp_dir: READ/WRITE; cp_dlab: 0/1; cp_access_kind: NORMAL/DEBUG/UNMAPPED; cp_dbg_core_state; cp_dbg_lsr_err_pending: 0/1; cp_fifo_count: 0, intermedia, 16; cp_init_seq: completa; cp_ier_reset y cp_ier_reserved: comprobados.*	*cp_addr × cp_dir × cp_dlab: bins legales; CE-20 añade 8, 12 y no mapeadas con ambos DLAB (no mapeadas: exploratorias). cp_addr[8] × cp_dbg_lsr_err_pending; cp_addr[12] × cp_fifo_count; cp_addr[8,12] × cp_dbg_core_state (solo READ); cp_addr[7] × cp_dir.*	*Debug 1/2 (8 y 12) solo lectura con wb_sel_i=1111 y variante de 32 bits; no exigir cp_dir=WRITE para Debug. Direcciones no mapeadas: cobertura exploratoria, sin respuesta funcional impuesta. No exigir accesos de lectura/escrituras incompatibles con los registros. Valores incorrectos de reset/reservados son fallos, no bins-meta.*
cg_bus	*Monitor Wishbone, las operaciones se contabilizan al recibir ACK y los retardos se miden mediante los cambios observados en cyc y stb*	*Cp_sel,cp_dir, cp_sel_ilegal, cp_ack_latency, cp_inter_transaction_gap,cp_stb_delay*	*Cp_sel x cp_dir , cp_sel_illegal x cp_dir*	*Se excluyen combinanciones no definidas de wb_sel_i .*
cg_rst	*La observación de la activación de wb_rst_if, el contexto lo obtiene de los monitores Wishbone/serie*	*Cp_reset_phase, cp_reset_type*	*cp_reset_type × cp_reset_phase*	*Se ignoran las combinaciones de reset inicial porque el DUT todavía no ha iniciado operación.*
cg_line_cfg	*La observación de escrituras completadas a LCR y al divisor mediante el monitor Wishbone. Los cruces con TX/RX se muestrean al observar tramas serie*	*cp_wls, cp_stb, cp_parity, cp_bc, cp_dl, cp_dir*	*cp_wls × cp_dir; cp_stb × cp_wls; cp_parity × cp_dir; cp_wls × cp_stb × cp_dir; cp_dl × cp_wls*	*Con PEN = 0, EPS y SP se agrupan en el modo NONE porque no afectan la trama.*
cg_fifo_cfg	*A partir de las escrituras válidas a FCR observadas por el monitor Wishbone y eventos proporcionados por los monitores serie y el modelo de referencia*	*Cp_tl, cp_rx_clr, cp_tx_clr, cp_fcr_b0, cp_fcr_b5_3, cp_rx_busy y cp_tx_busy*	*Cp_tl x cp_rx_fifo_level, cp_rx_clr x cp_rx_busy, cp_tx_clr x cp_tx_busy, cp_fcr_b0 x cp_tx_fifo_level*	*Todos los valores de FCR son legales, no se exige una lectura funcional de FCR porque la dirección 2 devuelve IIR*
cg_tx	*A partir de escrituras de THR observadas por Whisbone, transiciones por el monitor serie y la ocupación estimada por el modelo de FIFO TX*	*Cp_tx_fifo_level , cp_tx_busy, cp_lsr_thre, cp_lsr_temt, cp_overflow, cp_tx_burst_len*	*Cp_tx_fifo_level x cp_tx_busy, cp_lsr_thre x cp_tx_fifo_level, cp_lsr_temt × cp_tx_busy , cp_tx_fifo_level[16] × cp_overflow , TX x WLS*	*Se excluyen ocupaciones mayores que 16 por exceder la profundidad de FIFO.*
cg_rx	*Alimentado por el monitor serie RX, el monitor Wishbone y el*	*cp_rx_fifo_level, cp_rx_event ,cp_rbr_read,*	*cp_rx_fifo_level × cp_wls (17 × 4 = 68 bins); cp_rx_fifo_level*	*Ocupaciones mayores que 16 son ilegales. Las lecturas de*

Covergroup	Dónde se muestrea	Coverpoints	Cruces	Bins ilegales o ignorados (justificación)
	modelo de referencia de la FIFO RX.	*cp_rx_busy, cp_wls*	*× cp_rx_event para llegadas y extracciones; cp_rbr_read × cp_rx_fifo_level; cp_rx_fifo_level × cp_rx_busy.*	*dirección 0 con DLAB = 1 corresponden a DLL, no a RBR.*
cg_err	*Alimentado por el monitor serie RX, monitor Wishbone y modelo de FIFO RX. Se muestrea al recibir caracteres, detectar break u overrun y observar lecturas válidas de LSR.*	*cp_oe, cp_pe,cp_fe, cp_bi, cp_lsr7, cp_err_in_fifo, cp_error_kind, cp_break_len, cp_stop2_err*	*cp_oe × cp_tl; cp_pe × cp_parity; cp_fe × cp_wls; cp_bi × cp_break_len; cp_lsr7 × cp_err_in_fifo; cp_lsr7 × cp_error_kind.*	*Se ignora la inyección de PE cuando la paridad ecp_oe × cp_tl; cp_pe × cp_parity; cp_fe × cp_wls; cp_bi × cp_break_len; cp_lsr7 × cp_err_in_fifo; cp_lsr7 × cp_error_kind.está deshabilitada y la corrupción de una segunda porción de parada cuando STB = 0.*
cg_ser	*Alimentado por el monitor serie RX y los resultados del scoreboard. Se muestrea al observar glitches en reposo, completar tramas con desviación de baudios y comprobar la recuperación posterior.*	*cp_baud_dev, cp_glitch, cp_glitch_width, cp_wls, cp_rx_result*	*cp_baud_dev × cp_wls; cp_baud_dev × cp_rx_result para caracterización; cp_glitch × cp_glitch_width cuando existe glitch; cp_glitch_width × cp_wls durante recepción posterior.*	*Se ignoran los casos con DL= 0 y loopback activo para perturbaciones del pin RX externo. El ancho de glitch se registra únicamente cuando existe un pulso de 1–7 ticks.*
cg_reconfig	*Utilizando transacciones completadas del monitor Wishbone y el estado de actividad informado por los monitores serie y el modelo de referencia.*	*cp_cfg_phase, cp_reconfig_target, cp_traffic_state, cp_dl_order, cp_lcr_field, cp_dl_transition*	*cp_reconfig_target × cp_traffic_state durante reconfiguración; cp_dl_order × cp_cfg_phase; cp_lcr_field × cp_traffic_state para cambios de formato.*	*Las operaciones interrumpidas por reset no se cuentan. Las escrituras que repiten el valor anterior no cuentan como cambios efectivos, salvo acciones de control como limpieza de FIFO.*
cg_modem	*Alimentado los tres monitores principales. Se muestrea ante transiciones de las entradas, escrituras a MCR, lecturas de MSR, limpieza de deltas, actividad serie en loopback y reset*	*cp_signal, cp_level , cp_transition , cp_delta , cp_delta_type , cp_clear_event , cp_dtr_cfg, cp_rts_cfg, cp_dtr_pin, cp_rts_pin , cp_mcr_cfg , cp_loopback ,cp_modem_signal , cp_mcr_reset , cp_serial_activity*	*cp_signal × cp_level; cp_signal × cp_transition × cp_delta; cp_cts_edge × cp_dcts; cp_dsr_edge × cp_ddsr; cp_ri_edge × cp_teri; cp_dcd_edge × cp_ddcd; cp_delta_type × cp_clear_event; cp_dtr_cfg × cp_dtr_pin; cp_rts_cfg × cp_rts_pin; cp_loopback × cp_out1; cp_loopback × cp_out2; cp_loopback × cp_modem_signal × cp_level; cp_loopback × cp_serial_activity.*	*No se exige leer MCR mediante acceso normal, porque es de solo escritura. Durante loopback, MSR[7:4] debe relacionarse con MCR y no con las entradas físicas.*
cg_int	*Alimentado por los*	*cp_ier_mask,*	*cp_ier_mask ×*	*Se excluyen*

Covergroup	Dónde se muestrea	Coverpoints	Cruces	Bins ilegales o ignorados (justificación)
	monitores del ambiente. Se muestrea al observar fuentes habilitadas/enmascaradas, estados estables de IIR/int_o, coincidencias de fuentes y servicios completados.	*cp_irq_source, cp_ier_enable, cp_irq_state, cp_pending, cp_iir_pending, cp_iir_id, cp_priority_pair, cp_rx_error, cp_fifo_trigger, cp_fifo_below_trigger, cp_thre_event, cp_modem_source cp_service_action (NONE, READ_LSR, READ_RBR, READ_IIR, WRITE_THR, READ_MSR); cp_pending_after (NO, YES); cp_service_mode (POLLING, INTERRUPT); cp_reconfig (0, 1); cp_iir_reset y cp_iir_fixed_bits.*	*cp_irq_source × cp_irq_state; cp_irq_source × cp_ier_enable; cp_irq_state × cp_pending; cp_iir_pending × cp_irq_state; cp_priority_pair × cp_iir_id; cp_rx_error × cp_ier_rls y cp_rx_error × cp_iir_id[RLS]; cp_fifo_trigger × cp_iir_id[RDA/TIMEOUT]; cp_fifo_trigger × cp_fifo_below_trigger × cp_pending_after; cp_thre_event × cp_ier_enable; cp_modem_source × cp_iir_id[MODEM]; cp_iir_id × cp_service_action × cp_pending_after; cp_irq_state × cp_reconfig × cp_service_mode.*	*condiciones inválidas o enmascaradas. Cada interrupción requiere su servicio correspondiente, respetando prioridades y fuentes pendientes.*

14. Cobertura de código

Métrica	Jerarquía incluida	Meta	Exclusiones previstas y justificación
Línea	tb_top.dut y toda la jerarquía RTL del UART 16550, incluyendo uart_top, uart_wb, uart_regs, uart_receiver, uart_transmitter, uart_rfifo, uart_tfifo, uart_debug_if, uart_sync_flops y las memorias raminfr instanciadas.	≥ 95 %	No se excluyen elementos inicialmente. Solo se excluirá código cuya inalcanzabilidad sea demostrada y documentada mediante URG.
Condición	Misma jerarquía RTL del DUT. Incluye las expresiones condicionales de control, registros, FIFOs, transmisión, recepción e interrupciones.	≥ 95 %	No se excluyen condiciones únicamente porque sean difíciles de alcanzar. Si el reporte identifica resultados booleanos imposibles debido a constantes, parámetros o restricciones estructurales de la configuración compilada.
Rama	Misma jerarquía RTL del DUT, considerando las alternativas de las estructuras if, else y case instrumentadas por VCS.	≥ 95 %	Se intentará cubrir todas las ramas alcanzables mediante la prueba base, semillas y plusargs. Solo podrán excluirse ramas inalcanzables bajo la configuración del DUT y la especificación del proyecto.
FSM	Máquinas de estados reconocidas por las herramientas dentro de tb_top.dut. principalmente las relacionadas con los procesos de transmisión, recepción y control interno.	≥ 95 %	Se buscará cubrir todos los estados y transiciones alcanzables de las FSM reconocida.
Toggle	Señales y registros instrumentado dentro de tb_top.dut y sus submódulos, incluyendo señales de control, contadores, registros y lógica de las FIFOS.	≥ 95 %	Podrán proponerse exclusiones para señales constante o incapaces de realizar alguna transición en la configuración seleccionada.

15. Criterios de cierre

Criterio	Meta	Cómo se mide
Cobertura funcional	≥ 95 %	Reporte de cobertura del simulador, con los resultados obtenidos de todas las semillas.
Cada métrica de cobertura de código	≥ 95 %	Reporte de cobertura de código del simulador sobre el RTL del DUT, combinado en toda la regresión y revisado por métrica.
UVM_ERROR en la regresión	0, salvo errores del DUT documentados	Resumen de severidades que imprime UVM al final de cada simulación (report summary).
Aserciones con cover alcanzado	100 %	Reporte de cobertura de aserciones del simulador: cada cover property tiene al menos un acierto en la regresión combinada y ningún assert falla.

16. Riesgos y mitigación

Riesgo	Impacto	Mitigación
Simulaciones muy lentas con divisores grandes	Alto	Usar divisores pequeños por defecto y dejar los grandes para una prueba aparte.
Mala interpretación del RTL	Alto	Documentar cada discrepancia, decidir qué se espera y confirmarlo simulando.
Errores en el modelo del scoreboard que generen fallas falsas	Alto	Validar el modelo por partes con pruebas dirigidas antes de usar pruebas aleatorias.
El modelo de registros (RAL) espera un valor distinto al real porque algunos registros	Medio	Indicarle al RAL cuáles registros cambian por su cuenta y no incluirlos en las pruebas

Riesgo	Impacto	Mitigación
cambian solos o al leerlos		automáticas de lectura y escritura
Casos difíciles de alcanzar (overrun, timeout, break)	Medio	Crear secuencias específicas para provocarlos y revisar los huecos de cobertura.
Problemas al integrar las tres ramas	Medio	Acordar desde el inicio las transacciones y los nombres, e integrar seguido.
Errores que no se pueden reproducir	Medio	Guardar la semilla y los plusargs de cada simulación.

17. Distribución del trabajo

Cada integrante es responsable principal de algunos componentes, pero los tres deben entender todo el ambiente (entrevista individual, sección 8.2 del enunciado).

Componente o tarea	Responsable principal	Revisor (otro integrante)	Semana
Lectura de la especificación y compilación del RTL en VCS	Todos	—	S1
Repositorio Git y acceso del profesor	Randy	Eric	S1
Plan de pruebas (E1)	Todos	—	S2
Rama Wishbone: wb_if, wb_item, wb_driver, wb_monitor, wb_sequencer, wb_agent y wb_agent_cfg	Eric	Diego	S3
Rama Wishbone: secuencias de lectura y escritura	Eric	Diego	S3
Rama Wishbone: adaptador del RAL (reg2bus / bus2reg)	Eric	Diego	S3
Rama Wishbone: aserciones SVA de Wishbone	Eric	Diego	S4
Rama serie: uart_ser_if, uart_ser_item, uart_ser_sequencer, uart_ser_agent y uart_ser_agent_cfg	Diego	Randy	S3
Rama serie: driver con inyección de errores (paridad, trama, break, glitch, baudios)	Diego	Randy	S3
Rama serie: monitores de stx y srx	Diego	Randy	S3
Rama serie: secuencias de tráfico RX	Diego	Randy	S4
Rama serie: aserciones SVA de serie y loopback	Diego	Randy	S4
Rama módem: modem_if, modem_item, modem_driver, modem_monitor, modem_sequencer, modem_agent y modem_agent_cfg	Randy	Eric	S3
Rama módem: secuencias de módem	Randy	Eric	S4
Rama módem: monitor de interrupción (int_o)	Randy	Eric	S4
Rama módem: aserciones SVA de módem	Randy	Eric	S4
tb_top, paquete uart_pkg e integración de las interfaces	Todos	—	S3
uart_env y uart_env_cfg	Todos	—	S3
uart_base_test y parse_plusargs()	Todos	—	S3

Componente o tarea	Responsable principal	Revisor (otro integrante)	Semana
Modelo RAL (reg_block, mapas DLAB, puerta trasera) y predictor	Todos	—	S3
Secuenciador virtual y secuencia virtual principal	Todos	—	S4
Secuencias de registros y reconfiguración sobre el RAL	Todos	—	S4
Scoreboard y modelo de referencia	Todos	—	S4
Secuencias integradas del RAL y exclusiones	Todos	—	S5
Covergroups de cobertura funcional	Todos	—	S5
Script de regresión, fusión con urg y curva de línea base	Todos	—	S5
Análisis de huecos, archivos .args y exclusiones .el	Todos	—	S6
Reporte de errores del DUT	Todos	—	S6
Informe final y presentación	Todos	—	Final

18. Anexos

```
UVM_INFO @ 0: reporter [RNTST] Running test uart_hello_test...
UVM_INFO uart_hello_pkg.sv(71) @ 350000: uvm_test_top [HELLO] LCR = 0x03, IIR = 0xc1, LSR = 0x60
UVM_INFO uart_hello_pkg.sv(81) @ 350000: uvm_test_top [HELLO] instancia correcta: el DUT responde por wishbone
UVM_INFO /apps/vcsmx/vcs/X-2025.06-SP1//etc/uvm-1.2/src/base/uvm_objection.svh(1270) @ 550000: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
UVM_INFO /apps/vcsmx/vcs/X-2025.06-SP1//etc/uvm-1.2/src/base/uvm_report_server.svh(904) @ 550000: reporter [UVM/REPORT/SERVER]
--- UVM Report Summary ---

** Report counts by severity
UVM_INFO :    5
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
** Report counts by id
[HELLO]     2
[RNTST]     1
[TEST_DONE]     1
[UVM/RELNOTES]     1

$finish called from file "/apps/vcsmx/vcs/X-2025.06-SP1//etc/uvm-1.2/src/base/uvm_root.svh", line 532.
$finish at simulation time              550000
"uart_wb_sva.sv", 38: th_top.dut.u_wb_sva.c_master_wait, 28 attempts, 1 match
           V C S   S i m u l a t i o n   R e p o r t
Time: 550000 ps
CPU Time:      0.560 seconds;       Data structure size:   0.3Mb
Tue Oct 20 22:13:30 2026
Done
```